Club de Algoritmos de Sinaloa Notebook

Contents

1 templates 2
- 1.1 Template C++ . . . 2
- 1.2 Stress Test . . . 2

2 DataStructure 3
- 2.1 DSU Rollback . . . 3
- 2.2 Dynamic Li Chao Tree . . . 3
- 2.3 Fenwick Tree . . . 3
- 2.4 Fenwick Tree 2D . . . 3
- 2.5 Fraction . . . 3
- 2.6 Li Chao Tree . . . 4
- 2.7 Line Container . . . 4
- 2.8 Matrix . . . 4
- 2.9 Mo Queries . . . 4
- 2.10 Order Statistics Tree . . . 5
- 2.11 Segment Tree . . . 5
- 2.12 Segment Tree 2D . . . 5
- 2.13 Segment Tree Iterative . . . 5
- 2.14 Segment Tree Lazy . . . 5
- 2.15 Segment Tree Persistent . . . 6
- 2.16 Segment Tree Sparse . . . 6
- 2.17 Sparse Table . . . 6
- 2.18 Sparse Table Linear . . . 6
- 2.19 Sub Matrix . . . 6
- 2.20 Treap . . . 7

3 NumberTheory 8
- 3.1 CRT . . . 8
- 3.2 Discrete Log . . . 8
- 3.3 Euclid . . . 8
- 3.4 Linear Diophantine . . . 8
- 3.5 Mod Inverses . . . 8
- 3.6 Modint . . . 8
- 3.7 Multiplicative Functions . . . 9
- 3.8 Primitive Root . . . 9
- 3.9 Sieve . . . 9

4 Combinatorial 10
- 4.1 Catalan . . . 10
- 4.2 Combinations . . . 10

5 Numerical 11
- 5.1 Determinant . . . 11
- 5.2 FFT . . . 11
- 5.3 Gauss . . . 11
- 5.4 Simplex . . . 11
- 5.5 Simpson . . . 12
- 5.6 Xor Basis . . . 12

6 Graphs 13
- 6.1 2SAT . . . 13
- 6.2 Biconnected Components . . . 13
- 6.3 Bridges And Articulation Points . . . 13
- 6.4 Dominator Tree . . . 13
- 6.5 Maximal Cliques . . . 14
- 6.6 Maximum Clique . . . 14
- 6.7 SCC . . . 14
- 6.8 General Matching . . . 14
- 6.9 Hopcroft Karp . . . 15
- 6.10 Hungarian . . . 15
- 6.11 Kuhn . . . 15
- 6.12 Minimum Vertex Cover . . . 15
- 6.13 Directed MST . . . 16
- 6.14 Kruskal . . . 16
- 6.15 Prim . . . 16
- 6.16 Hierholzer . . . 16
- 6.17 Topological Sort . . . 16
- 6.18 Dinic . . . 17
- 6.19 Min Cost Max Flow . . . 17
- 6.20 Push Relabel . . . 17
- 6.21 Bellman-Ford . . . 18
- 6.22 Dijkstra . . . 18
- 6.23 Floyd-Warshall . . . 18
- 6.24 Centroid Decomposition . . . 18
- 6.25 Euler Tour . . . 18
- 6.26 HLD . . . 18
- 6.27 LCA . . . 19

7 Strings 20
- 7.1 Aho Corasick . . . 20
- 7.2 Dynamic Aho Corasick . . . 20
- 7.3 Evil Hashing . . . 21
- 7.4 Hashing . . . 21
- 7.5 Hashing Dynamic . . . 21
- 7.6 KMP . . . 22
- 7.7 KMPAutomaton . . . 22

7.8 Manacher . . . 22
7.9 Palindromic Tree . . . 22
7.10 Suffix Array . . . 23
7.11 Suffix Automaton . . . 23
7.12 Suffix Tree . . . 23
7.13 Trie . . . 24
7.14 ZAlgorithm . . . 24

8 Geometry 25
8.1 Coords . . . 25
8.2 Hull3D . . . 25
8.3 Line3D . . . 25
8.4 Plane . . . 25
8.5 Point3D . . . 26
8.6 Polyhedron . . . 26
8.7 Sphere . . . 26
8.8 Circle . . . 26
8.9 Closest Pair . . . 27
8.10 Convex Hull . . . 27
8.11 Half Plane . . . 27
8.12 Line . . . 28
8.13 Minimum Enclosing Circle . . . 28
8.14 Minkowski Sum . . . 28
8.15 Point . . . 28
8.16 Polar Sort . . . 29
8.17 Polygon . . . 29
8.18 Segment . . . 29

9 Extras 31
9.1 Custom Hash . . . 31
9.2 Dates . . . 31
9.3 Iterate Submasks . . . 31
9.4 LIS . . . 31
9.5 Random . . . 31

1 templates

1.1 Template C++

```cpp
#include <bits/stdc++.h>
using namespace std;
// Pura Gente del Coach Moy

using ll = long long;
using pi = pair<int, int>;
using vi = vector<int>;

#define pb push_back
#define SZ(x) ((int)(x).size())
#define ALL(x) begin(x), end(x)
#define FOR(i, a, b) for (int i = (int)a; i < (int)b; ++i)
#define ROF(i, a, b) \
  for (int i = (int)a - 1; i >= (int)b; --i)
#define ENDL '\n'

int main() { cin.tie(0)->sync_with_stdio(0); }
```

1.2 Stress Test

```
# Script de stress testing para verificar soluciones contra
      una solucion de fuerza bruta
# mediante generacion aleatoria de casos de prueba.

set -e
source ~/.bashrc

g++-14 -Wall -Wconversion -Wfatal-errors -g -std=c++23 -
      fsanitize=undefined,address sol.cpp -o sol
g++-14 -Wall -Wconversion -Wfatal-errors -g -std=c++23 -
      fsanitize=undefined,address brute.cpp -o brute
g++-14 -Wall -Wconversion -Wfatal-errors -g -std=c++23 -
      fsanitize=undefined,address gen.cpp -o gen
# g++-14 -Wall -Wconversion -Wfatal-errors -g -std=c++23 -
      fsanitize=undefined,address checker.cpp -o checker

for((i = 1; ; ++i)); do
    ./gen $i > input_file
    ./sol < input_file > myAnswer
    ./brute < input_file > correctAnswer

    # Validacion por respuesta exacta (diff) o por
         verificador (checker)
    diff -Z myAnswer correctAnswer > /dev/null || break
    # ./checker > checker_log || break

    echo "Passed test: "  $i
done

echo "WA on the following test:"
cat input_file
echo "Your answer is:"
cat myAnswer
echo "Correct answer is:"
cat correctAnswer
# cat checker_log
```

2 DataStructure

2.1 DSU Rollback

```
/**
 * Descripcion: Estructura de conjuntos disjuntos con la
 * capacidad de regresar a estados anteriores.
 * Si no es necesario, ignorar st, time() y rollback().
 *
 * Uso: int t = uf.time(); ...; uf.rollback(t)
 *
 * Tiempo: O(log n)
 * Status: testeadisimo
 */

struct RollbackDSU {
  vector<int> e;
  vector<pi> st;
  void init(int n) { e = vi(n, -1); }
  int size(int x) { return -e[get(x)]; }
  int get(int x) {
    return e[x] < 0 ? x : e[x] = get(e[x]);
  }
  int time() { return st.size(); }
  void rollback(int t) {
    for (int i = time(); i-- > t;)
      e[st[i].first] = st[i].second;
    st.resize(t);
  }
  bool join(int a, int b) {
    a = get(a), b = get(b);
    if (a == b) return false;
    if (e[a] > e[b]) swap(a, b);
    st.push_back({a, e[a]});
    st.push_back({b, e[b]});
    e[a] += e[b];
    e[b] = a;
    return true;
  }
};
```

2.2 Dynamic Li Chao Tree

```
/*
 * Descripcion: El Arbol de Li-Chao es una estructura de
 * datos utilizada en algoritmos de programacion dinamica
 * y geometrica para realizar consultas de maximo (o
 * minimo) en un conjunto de puntos en una linea (o un
 * plano). Tiempo : Construccion : O(N log N) Insercion y
 * Consultas : O(log N) Alternativa, construcion dinamica
 * mas eficiente.
 * **Cuidado con la memoria dinamica pude causar errores
 * si no se tiene cuidado.
 * **Recomendacion: No hacer declaraciones globales
 */

struct Line {
  ll m, c;
  ll eval(int x) { return m * x + c; }
};
struct node {
  Line line;
  node* left = nullptr;
  node* right = nullptr;
  node(Line line) : line(line) {}
  void add_segment(Line nw, ll l, ll r, ll L, ll R) {
    if (l > r || r < L || l > R) return;
    ll m = (l + 1 == r ? l : (l + r) / 2);
    if (l >= L and r <= R) {
      bool lef = nw.eval(l) < line.eval(l);
      bool mid = nw.eval(m) < line.eval(m);
      if (mid) swap(line, nw);
      if (l == r) return;
      if (lef != mid) {
        if (left == nullptr)
          left = new node(nw);
        else
          left->add_segment(nw, l, m, L, R);
      } else {
        if (right == nullptr)
          right = new node(nw);
        else
          right->add_segment(nw, m + 1, r, L, R);
      }
      return;
    }
    if (max(l, L) <= min(m, R)) {
      if (left == nullptr) left = new node({0, inf});
      left->add_segment(nw, l, m, L, R);
    }
    if (max(m + 1, L) <= min(r, R)) {
      if (right == nullptr) right = new node({0, inf});
      right->add_segment(nw, m + 1, r, L, R);
    }
  }
  ll query_segment(ll x, ll l, ll r, ll L, ll R) {
    if (l > r || r < L || l > R) return inf;
    ll m = (l + 1 == r ? l : (l + r) / 2);
    if (l >= L and r <= R) {
      ll ans = line.eval(x);
      if (l < r) {
        if (x <= m && left != nullptr)
          ans = min(ans,
                    left->query_segment(x, l, m, L, R));
        if (x > m && right != nullptr)
          ans = min(ans, right->query_segment(x, m + 1, r,
                                              L, R));
      }
      return ans;
    }
    ll ans = inf;
    if (max(l, L) <= min(m, R)) {
      if (left == nullptr) left = new node({0, inf});
      ans = min(ans, left->query_segment(x, l, m, L, R));
    }
    if (max(m + 1, L) <= min(r, R)) {
      if (right == nullptr) right = new node({0, inf});
      ans = min(ans,
                right->query_segment(x, m + 1, r, L, R));
    }
    return ans;
  }
};

struct LiChaoTree {  // the input values for lichaotree
                     // are boundaries of x values you can
                     // use to query with
  int L, R;
  node* root;
  LiChaoTree()
      : L(numeric_limits<int>::min() / 2),
        R(numeric_limits<int>::max() / 2),
        root(nullptr) {}
  LiChaoTree(int L, int R) : L(L), R(R) {
    root = new node({0, inf});
  }
  void add_line(Line line) {
    root->add_segment(line, L, R, L, R);
  }
  // y = mx + b: x in [l, r]
  void add_segment(Line line, int l, int r) {
    root->add_segment(line, L, R, l, r);
  }
  ll query(int x) {
    return root->query_segment(x, L, R, L, R);
  }
  ll query_segment(int x, int l, int r) {
    return root->query_segment(x, l, r, L, R);
  }
};
```

2.3 Fenwick Tree

```
/**
 * Descripcion: arbol binario indexado, util para
 * consultas en donde es posible hacer
 * inclusion-exclusion, suma, multiplicacion, etc.
 * Tiempo:
 * O(log n)
 */

struct FT {
  vector<ll> s;
  FT(int n) : s(n) {}
  void update(int pos, ll dif) {  // a[pos] += dif
    for (; pos < SZ(s); pos |= pos + 1) s[pos] += dif;
  }
  ll query(int pos) {  // sum of values in [0, pos)
    ll res = 0;
    for (; pos > 0; pos &= pos - 1) res += s[pos - 1];
    return res;
  }
  int lower_bound(
      ll sum) {  // min pos st sum of [0, pos] >= sum
    // Returns n if no sum is >= sum, or -1 if empty sum
    // is.
    if (sum <= 0) return -1;
    int pos = 0;
    for (int pw = 1 << 25; pw; pw >>= 1)
      if (pos + pw <= SZ(s) && s[pos + pw - 1] < sum)
        pos += pw, sum -= s[pos - 1];
    return pos;
  }
};
```

2.4 Fenwick Tree 2D

```
/**
 * Descripcion: arbol binario indexado 2D, util para
 * consultas en un espacio 2D como una matriz
 * Tiempo:
 * Construir el BIT: O(NM log(N)*log(M))
 * Consultas y Actualizaciones: O(log(N)*log(M))
 */

int ft[MAX + 1][MAX + 1];
void upd(int i0, int j0, int v) {
  for (int i = i0 + 1; i <= MAX; i += i & -i)
    for (int j = j0 + 1; j <= MAX; j += j & -j)
      ft[i][j] += v;
}
int get(int i0, int j0) {
  int r = 0;
  for (int i = i0; i; i -= i & -i)
    for (int j = j0; j; j -= j & -j) r += ft[i][j];
  return r;
}
int get_sum(int i0, int j0, int i1, int j1) {
  return get(i1, j1) - get(i1, j0) - get(i0, j1) +
         get(i0, j0);
}
```

2.5 Fraction

```
/**
 * Descripcion: estructura para manejar fracciones.
 * Se asume que el denominador no es cero.
 * Tiempo por operacion: O(1)
 */
struct Frac {
```

```
  int num, den;

  Frac(int num, int den) {
    int g = gcd(num, den);
    this->num = num / g;
    this->den = den / g;
  }

  Frac operator*(Frac& f) const {
    return Frac(num * f.num, den * f.den);
  }
  Frac operator/(Frac& f) const {
    return (*this) * Frac(f.den, f.num);
  }
  Frac operator+(Frac& f) const {
    return Frac(num * f.den + den * f.num, den * f.den);
  }
  Frac operator-(Frac& f) const {
    return Frac(num * f.den - den * f.num, den * f.den);
  }
  bool operator<(Frac& other) const {
    return num * other.den < other.num * den;
  }
  bool operator==(Frac& other) const {
    return num == other.num && den == other.den;
  }
  bool operator!=(Frac& other) const {
    return !(*this == other);
  }
};
```

2.6 Li Chao Tree

```
/*
 * Descripcion: El Arbol de Li-Chao es una estructura de
 * datos utilizada en algoritmos de programacion dinamica
 * y geometrica para realizar consultas de maximo (o
 * minimo) en un conjunto de puntos en una linea (o un
 * plano). Tiempo : Construccion : O(N log N) Insercion y
 * Consultas : O(log N)
 */

class LiChao {
  vector<ll> m, b;
  int n, sz;
  ll *x;
#define gx(i) (i < sz ? x[i] : x[sz - 1])
  void update(int t, int l, int r, ll nm, ll nb) {
    ll xl = nm * gx(l) + nb, xr = nm * gx(r) + nb;
    ll yl = m[t] * gx(l) + b[t], yr = m[t] * gx(r) + b[t];
    if (yl >= xl && yr >= xr) return;
    if (yl <= xl && yr <= xr) {
      m[t] = nm, b[t] = nb;
      return;
    }
    int mid = (l + r) / 2;
    update(t << 1, l, mid, nm, nb);
    update(1 + (t << 1), mid + 1, r, nm, nb);
  }

 public:
  LiChao(ll *st, ll *en) : x(st) {
    sz = int(en - st);
    for (n = 1; n < sz; n <<= 1);
    m.assign(2 * n, 0);
    b.assign(2 * n, -INF);
  }
  void insert_line(ll nm, ll nb) {
    update(1, 0, n - 1, nm, nb);
  }
  ll query(int i) {
    ll ans = -INF;
    for (int t = i + n; t; t >>= 1)
      ans = max(ans, m[t] * x[i] + b[t]);
    return ans;
  }
};
```

2.7 Line Container

```
/*
 * Line Container (Convex Hull Trick)
 * Descripcion: Contenedor donde puedes anadir lineas en
 * forma kx+m, y hacer consultas al valor maximo en un
 * punto x. Pro-Tip: Si se busca el valor minimo en un
 * punto x, anadir las lineas con pendiente K negativa (la
 * consulta se dara de forma negativa)
 * Tiempo: O(log n)
 */

struct Line {
  mutable ll k, m, p;
  bool operator<(const Line& o) const { return k < o.k; }
  bool operator<(ll x) const { return p < x; }
};

struct LineContainer : multiset<Line, less<>> {
  // (para doubles, usar inf = 1/.0, div(a,b) = a/b)
  static const ll inf = LLONG_MAX;
  ll div(ll a, ll b) {  // floored division
    return a / b - ((a ^ b) < 0 && a % b);
  }
  bool isect(iterator x, iterator y) {
    if (y == end()) return x->p = inf, 0;
    if (x->k == y->k)
      x->p = x->m > y->m ? inf : -inf;
    else
      x->p = div(y->m - x->m, x->k - y->k);
    return x->p >= y->p;
  }
  void add(ll k, ll m) {
    auto z = insert({k, m, 0}), y = z++, x = y;
    while (isect(y, z)) z = erase(z);
    if (x != begin() && isect(--x, y))
      isect(x, y = erase(y));
    while ((y = x) != begin() && (--x)->p >= y->p)
      isect(x, erase(y));
  }
  ll query(ll x) {
    assert(!empty());
    auto l = *lower_bound(x);
    return l.k * x + l.m;
  }
};
```

2.8 Matrix

```
/**
 * Descripcion: estructura de matriz con algunas
 * operaciones basicas se suele utilizar para la
 * multiplicacion y/o exponenciacion de matrices
 * Aplicaciones:
 * Calcular el n-esimo fibonacci en tiempo logaritmico,
 * esto es posible ya que para la matriz M = {{1, 1},{1,
 * 0}}, se cumple que M^n = {{F[n + 1], F[n]}, {F[n], F[n
 * - 2]}} Dado un grafo, su matriz de adyacencia M, y otra
 * matriz P tal que P = M^k, se puede demostrar que
 * P[i][j] contiene la cantidad de caminos de longitud k
 * que inician en el i-esimo nodo y terminan en el
 * j-esimo.
 * Tiempo: O(n^3 * log p) para la exponenciacion
 * y O(n^3) para la multiplicacion
 */

template <typename T>
struct Matrix {
  using VVT = vector<vector<T>>;
  VVT M;

  Matrix(VVT aux) : M(aux) {}

  Matrix operator*(Matrix& other) const {
    assert(SZ(M[0]) == SZ(other.M));
    int k = SZ(other.M[0]);
    VVT C(n, vector<T>(k, 0));
    FOR(i, 0, SZ(M))
    FOR(j, 0, k) FOR(l, 0, SZ(M[0])) C[i][j] =
        (C[i][j] + M[i][l] * other.M[l][j] % MOD) % MOD;
    return Matrix(C);
  }

  Matrix operator^(ll p) const {
    assert(p >= 0);
    Matrix ret(VVT(n, vector<T>(n))), B(*this);
    FOR(i, 0, n) { ret.M[i][i] = 1; }
    while (p) {
      if (p & 1) ret = ret * B;
      p >>= 1;
      B = B * B;
    }
    return ret;
  }

  void print() {
    cout << "--------------------" << ENDL;
    FOR(i, 0, n) {
      FOR(j, 0, m) { cout << M[i][j] << ' '; }
      cout << ENDL;
    }
    cout << "******************" << ENDL;
  }
};
```

2.9 Mo Queries

```
/*
 * Mos Algorithm
 * Descripcion: Es usado para responder consultas en
 * intervalos (L,R) de manera offline con base a un
 * orden especial basado en bloques moviendose de una
 * consulta a la siguiente anadiendo/removiendo puntos
 * en el inicio o el final. Rango de query inclusivo a
 * la izquierda y exclusivo a la derecha.
 *
 * Tiempo: O((N + Q) sqrt(N))
 */

void add(int idx, int end) {
}  // add a[idx] (end = 0 or 1)
void del(int idx, int end) {}  // remove a[idx]
int calc(){...}                // compute current answer

vi mosAlgo(vector<pi> Q) {
  // IMPORTANT!! blk ~= N/sqrt(Q)
  int L = 0, R = 0, blk = 350;
  vi s(SZ(Q)), res = s;
#define K(x) \
  pi(x.first / blk, x.second ^ -(x.first / blk & 1))
  iota(ALL(s), 0);
  sort(ALL(s),
       [&](int s, int t) { return K(Q[s]) < K(Q[t]); });
  for (int qi : s) {
    pi q = Q[qi];
    while (L > q.first) add(--L, 0);
    while (R < q.second) add(R++, 1);
    while (L < q.first) del(L++, 0);
    while (R > q.second) del(--R, 1);
    res[qi] = calc();
  }
  return res;
}
```

2.10 Order Statistics Tree

```cpp
/**
 * Descripcion: es una variante del BST, que ademas
 * soporta 2 operaciones extra ademas de insercion,
 * busqueda y eliminacion: Select(i) - find_by_order:
 * encontrar el i-esimo elemento (0-indexado) del conjunto
 * ordenado de los elementos, retorna un iterador. Rank(x)
 * - order_of_key: numero de elementos estrictamente
 * menores a x
 * Uso: oset<int> OST Funciona como un set,
 * por lo que nativamente no soporta elementos repetidos.
 * Si se necesitan repetidos, pero no eliminar valores,
 * cambiar la funcion comparadora por less_equal<T>. Si se
 * necesitan repetidos y tambien la eliminacion, agregar
 * una dimension a T en en donde el ultimo parametro sea
 * el diferenciador (por ejemplo, si estamos con enteros,
 * utilizar un pair donde el second sea unico). Modificar
 * el primer y tercer parametro (tipo y funcion
 * comparadora), si se necesita un mapa, en lugar de
 * null_type, escribir el tipo a mapear.
 * Tiempo: O(log n)
 */
#include <bits/extc++.h>
using namespace __gnu_pbds;

template <class T>
using oset = tree<T, null_type, less<T>, rb_tree_tag,
                  tree_order_statistics_node_update>;
```

2.11 Segment Tree

```cpp
/**
 * Descripcion: Segment tree para suma en rango y
 * actualizacion en punto. Rangos semiabiertos [l, r).
 *
 * Uso:
 * STree st(n);
 * st.upd(i, val);
 * int sum = st.query(l, r);
 *
 * Tiempo: O(\log n)
 */

#define NEUT 0

struct STree {
  int n;
  vi st;

  STree(int n) : n(n), st(4 * n) {}

  int query(int v, int tl, int tr, int l, int r) {
    if (tr <= l || r <= tl) return NEUT;
    if (l <= tl && tr <= r) return st[v];
    int tm = (tl + tr) / 2;
    return query(2 * v + 1, tl, tm, l, r) +
           query(2 * v + 2, tm, tr, l, r);
  }
  int query(int l, int r) { return query(0, 0, n, l, r); }

  void upd(int v, int tl, int tr, int i, int val) {
    if (tl + 1 == tr) {
      st[v] = val;
      return;
    }
    int tm = (tl + tr) / 2;
    if (i < tm)
      upd(2 * v + 1, tl, tm, i, val);
    else
      upd(2 * v + 2, tm, tr, i, val);
    st[v] = st[2 * v + 1] + st[2 * v + 2];
  }
  void upd(int i, int val) { upd(0, 0, n, i, val); }
};
```

2.12 Segment Tree 2D

```cpp
/**
 * Descripcion: Segment tree 2D para suma en un
 * rectangulo y actualizacion en punto.
 * Implementacion iterativa con 4x menos memoria.
 *
 * Uso:
 * STree st(n, m); st.build(a);
 * st.upd(x, y, v);
 * int ans = st.query(x0, y0, x1, y1);
 *
 * Tiempo: O(nm) construccion, O(\log n \log m) consulta y
 * actualizacion
 */

#define NEUT 0

struct STree {
  int n, m;
  vector<vi> st;
  STree(int n, int m)
      : n(n), m(m), st(2 * n, vi(2 * m, NEUT)) {}
  int comb(int x, int y) { return x + y; }
  void build(vector<vi>& a) {
    FOR(i, 0, n) FOR(j, 0, m) st[i + n][j + m] = a[i][j];
    FOR(i, 0, n)
    for (int j = m - 1; j; --j) st[i + n][j] =
        comb(st[i + n][j << 1], st[i + n][j << 1 | 1]);
    for (int i = n - 1; i; --i) FOR(j, 0, 2 * m)
    st[i][j] = comb(st[i << 1][j], st[i << 1 | 1][j]);
  }
  void upd(int x, int y, int v) {
    st[x + n][y + m] = v;
    for (int j = y + m; j > 1; j >>= 1)
      st[x + n][j >> 1] =
          comb(st[x + n][j], st[x + n][j ^ 1]);
    for (int i = x + n; i > 1; i >>= 1)
      for (int j = y + m; j; j >>= 1)
        st[i >> 1][j] = comb(st[i][j], st[i ^ 1][j]);
  }
  int query(int x0, int y0, int x1, int y1) {
    int r = NEUT;
    for (int i0 = x0 + n, i1 = x1 + n; i0 < i1;
         i0 >>= 1, i1 >>= 1) {
      int t[4], q = 0;
      if (i0 & 1) t[q++] = i0++;
      if (i1 & 1) t[q++] = --i1;
      FOR(k, 0, q)
      for (int j0 = y0 + m, j1 = y1 + m; j0 < j1;
           j0 >>= 1, j1 >>= 1) {
        if (j0 & 1) r = comb(r, st[t[k]][j0++]);
        if (j1 & 1) r = comb(r, st[t[k]][--j1]);
      }
    }
    return r;
  }
};
```

2.13 Segment Tree Iterative

```cpp
/**
 * Descripcion: Segment tree iterativo para consulta en
 * rango y actualizacion en punto. Los limites son
 * semiabiertos [l, r). Soporta cualquier operacion
 * asociativa en f.
 * Uso: STree st(n); st.upd(pos, val);
 * int ans = st.query(l, r);
 *
 * Tiempo: O(\log n) consulta y actualizacion, O(n)
 * memoria
 */

struct STree {
  using T = int;
  const T unit = 0;
  T f(T a, T b) { return a + b; }
  int n;
  vector<T> s;

  STree(int n = 0, T def = 0) : n(n), s(2 * n, def) {}

  void upd(int pos, T val) {
    for (s[pos += n] = val; pos /= 2;)
      s[pos] = f(s[pos * 2], s[pos * 2 + 1]);
  }

  T query(int b, int e) {
    T ra = unit, rb = unit;
    for (b += n, e += n; b < e; b /= 2, e /= 2) {
      if (b % 2) ra = f(ra, s[b++]);
      if (e % 2) rb = f(s[--e], rb);
    }
    return f(ra, rb);
  }
};
```

2.14 Segment Tree Lazy

```cpp
/**
 * Descripcion: Segment tree con lazy propagation para
 * suma y actualizacion en rango. Rangos semiabiertos [l,
 * r).
 * Uso: STree st(n); st.upd(l, r, val); int sum =
 * st.query(l, r);
 * Tiempo: O(\log n)
 */

#define NEUT 0

struct STree {
  int n;
  vi st, lazy;

  STree(int n) : n(n), st(4 * n), lazy(4 * n) {}

  void apply(int v, int tl, int tr, int val) {
    st[v] += val * (tr - tl);
    lazy[v] += val;
  }

  void push(int v, int tl, int tr) {
    if (lazy[v]) {
      int tm = (tl + tr) / 2;
      apply(2 * v + 1, tl, tm, lazy[v]);
      apply(2 * v + 2, tm, tr, lazy[v]);
      lazy[v] = 0;
    }
  }

  int query(int v, int tl, int tr, int l, int r) {
    if (tr <= l || r <= tl) return NEUT;
    if (l <= tl && tr <= r) return st[v];
    push(v, tl, tr);
    int tm = (tl + tr) / 2;
    return query(2 * v + 1, tl, tm, l, r) +
           query(2 * v + 2, tm, tr, l, r);
  }
  int query(int l, int r) { return query(0, 0, n, l, r); }

  void upd(int v, int tl, int tr, int l, int r, int val) {
    if (tr <= l || r <= tl) return;
    if (l <= tl && tr <= r) {
      apply(v, tl, tr, val);
```

```
      return;
    }
    push(v, tl, tr);
    int tm = (tl + tr) / 2;
    upd(2 * v + 1, tl, tm, l, r, val);
    upd(2 * v + 2, tm, tr, l, r, val);
    st[v] = st[2 * v + 1] + st[2 * v + 2];
  }
  void upd(int l, int r, int val) {
    upd(0, 0, n, l, r, val);
  }
};
```

2.15 Segment Tree Persistent

```
/**
 * Descripcion: Segment tree persistente para suma en
 * rango. Cada actualizacion genera una nueva raiz sin
 * modificar versiones previas. Rangos semiabiertos [l,
 * r).
 * Uso: STree st(n); int root1 = st.upd(root0, i,
 * val); int ans = st.query(root1, l, r);
 * Tiempo: O(\log
 * n)
 */

#define NEUT 0

struct STree {
  int n;
  vi lc, rc, val;

  int newNode(int l = 0, int r = 0, int v = NEUT) {
    lc.pb(l);
    rc.pb(r);
    val.pb(v);
    return SZ(val) - 1;
  }

  STree(int n) : n(n) { newNode(); }

  int query(int v, int tl, int tr, int l, int r) {
    if (!v || tr <= l || r <= tl) return NEUT;
    if (l <= tl && tr <= r) return val[v];
    int tm = (tl + tr) / 2;
    return query(lc[v], tl, tm, l, r) +
           query(rc[v], tm, tr, l, r);
  }
  int query(int root, int l, int r) {
    return query(root, 0, n, l, r);
  }

  int upd(int v, int tl, int tr, int i, int x) {
    int u = newNode(lc[v], rc[v], val[v]);
    if (tl + 1 == tr) {
      val[u] = x;
      return u;
    }
    int tm = (tl + tr) / 2;
    if (i < tm)
      lc[u] = upd(lc[u], tl, tm, i, x);
    else
      rc[u] = upd(rc[u], tm, tr, i, x);
    val[u] = val[lc[u]] + val[rc[u]];
    return u;
  }
  int upd(int root, int i, int x) {
    return upd(root, 0, n, i, x);
  }
};
```

2.16 Segment Tree Sparse

```
/**
 * Descripcion: Segment tree esparcido (dinamico) para
 * suma en rango y actualizacion en punto. Memoria O(q
 * \log n). Rangos semiabiertos [l, r).
 * Uso: STree st(n);
 * st.upd(i, val);
 * int sum = st.query(l, r);
 *
 * Tiempo: O(\log n)
 */

#define NEUT 0

struct STree {
  int n;
  vi lc, rc, val;

  int newNode() {
    lc.pb(0);
    rc.pb(0);
    val.pb(NEUT);
    return SZ(val) - 1;
  }

  STree(int n) : n(n) {
    newNode();
    newNode();
  }

  int query(int v, int tl, int tr, int l, int r) {
    if (!v || tr <= l || r <= tl) return NEUT;
    if (l <= tl && tr <= r) return val[v];
    int tm = (tl + tr) / 2;
    return query(lc[v], tl, tm, l, r) +
           query(rc[v], tm, tr, l, r);
  }
  int query(int l, int r) { return query(1, 0, n, l, r); }

  int upd(int v, int tl, int tr, int i, int x) {
    if (!v) v = newNode();
    if (tl + 1 == tr) {
      val[v] = x;
      return v;
    }
    int tm = (tl + tr) / 2;
    if (i < tm)
      lc[v] = upd(lc[v], tl, tm, i, x);
    else
      rc[v] = upd(rc[v], tm, tr, i, x);
    val[v] = val[lc[v]] + val[rc[v]];
    return v;
  }
  void upd(int i, int x) { upd(1, 0, n, i, x); }
};
```

2.17 Sparse Table

```
/**
 * Descripcion: util para consultas min/max en rango para
 * arreglos inmutables, ST[k][i] = min/max(A[i]...A[i +
 * 2^k - 1]);
 * Tiempo: O(n log n) en construccion y O(1)
 * por query
 */

template <class T>
struct SparseTable {
  vector<vector<T>> jmp;
  void init(const vector<T>& V) {
    if (SZ(jmp)) jmp.clear();
    jmp.emplace_back(V);
    for (int pw = 1, k = 1; pw * 2 <= SZ(V);
         pw *= 2, ++k) {
      jmp.emplace_back(SZ(V) - pw * 2 + 1);
      FOR(j, 0, SZ(jmp[k]))
        jmp[k][j] = min(jmp[k - 1][j], jmp[k - 1][j + pw]);
    }
  }
  T query(int l, int r) {  // [a, b)
    int dep = 31 - __builtin_clz(r - l);
    return min(jmp[dep][l], jmp[dep][r - (1 << dep)]);
  }
};
```

2.18 Sparse Table Linear

```
/**
 * Descripcion: RMQ en arreglo inmutable con construccion
 * lineal. Divide en bloques de B = 30; mask[i] guarda
 * (bit k -> posicion i - k) los candidatos a minimo de
 * las ventanas que terminan en i, asi el minimo de una
 * ventana de tamano <= B sale con un msb. Sobre los
 * minimos de los bloques se arma un sparse table normal.
 * Para maximo cambiar < por > en op.
 *
 * Uso:
 * RMQ<int> rmq(a);
 * int mn = rmq.query(l, r); // [l, r)
 * int pos = rmq.idx(l, r);
 *
 * Tiempo: O(n) construccion, O(1) consulta
 */

template <class T>
struct RMQ {
  static const int B = 30;
  int n, m;
  vector<T> v;
  vi mask, t;

  int op(int x, int y) { return v[x] < v[y] ? x : y; }
  int msb(int x) { return 31 - __builtin_clz(x); }
  // indice del minimo en [r - sz + 1, r], sz <= B
  int small(int r, int sz = B) {
    return r - msb(mask[r] & ((1 << sz) - 1));
  }

  RMQ(const vector<T>& a)
      : n(SZ(a)), m(n / B), v(a), mask(n), t(n) {
    for (int i = 0, at = 0; i < n; mask[i++] = at |= 1) {
      at = (at << 1) & ((1 << B) - 1);
      while (at && op(i, i - msb(at & -at)) == i)
        at ^= at & -at;
    }
    FOR(i, 0, m) t[i] = small(B * i + B - 1);
    for (int j = 1; (1 << j) <= m; j++)
      FOR(i, 0, m - (1 << j) + 1)
    t[m * j + i] =
        op(t[m * (j - 1) + i],
           t[m * (j - 1) + i + (1 << (j - 1))]);
  }

  int idx(int l, int r) {  // [l, r)
    if (r - l <= B) return small(r - 1, r - l);
    int x = l / B + 1, y = (r - 1) / B - 1;
    int ans = op(small(l + B - 1), small(r - 1));
    if (x > y) return ans;
    int j = msb(y - x + 1);
    return op(ans, op(t[m * j + x],
                      t[m * j + y - (1 << j) + 1]));
  }
  T query(int l, int r) { return v[idx(l, r)]; }
};
```

2.19 Sub Matrix

```
/*
```

```
 * Descripcion: Calcula la suma de una submatriz
 * rapidamente, dada la esquina superior izquierda e
 * inferior derecha
 * Uso: SubMatrix<int> m(matriz);
 * m.sum(0,0,2,2); // 4 elemento arriba a la izquierda
 *
 * Tiempo: O(N^2)
 */

template <class T>
struct SubMatrix {
  vector<vector<T>> p;
  SubMatrix(vector<vector<T>>& v) {
    int R = sz(v), C = sz(v[0]);
    p.assign(R + 1, vector<T>(C + 1));
    FOR(r, 0, R)
    FOR(c, 0, C)
    p[r + 1][c + 1] =
        v[r][c] + p[r][c + 1] + p[r + 1][c] - p[r][c];
  }
  T sum(int u, int l, int d, int r) {
    return p[d][r] - p[d][l] - p[u][r] + p[u][l];
  }
};
```

2.20 Treap

```
/**
 * Descripcion: poderoso arbol auto-balanceado.
 * Es un treap implicito, por lo que actua como
 * un contenedor dinamico secuencial que soporta
 * separaciones y uniones. Es facil de mejorar
 * con mas datos.
 *  Se usa 0-indexacion
 *  Tiene muchos ejemplos y atributos para
 *  suma y reverseo
 *  Rangos [l, r)
 *
 * Tiempo: O(log n)
 * Status: testeado en CSES Reversals and Sums
 */

struct Node {
  Node *l = 0, *r = 0;
  int val, y, c = 1;
  int s;
  bool rev;
  Node(int val)
      : val(val), y(rand()), s(val), rev(false) {}
  void recalc();
};

int sum(Node* n) { return n ? n->s : 0ll; }
int cnt(Node* n) { return n ? n->c : 0; }
void Node::recalc() {
  c = cnt(l) + cnt(r) + 1;
  s = val + sum(l) + sum(r);
}

void apply(Node* n) {
  if (n) {
    n->rev ^= 1;
    swap(n->l, n->r);
  }
}

void push(Node* n) {
  if (n && n->rev) {
    apply(n->l);
    apply(n->r);
    n->rev = false;
  }
}

pair<Node*, Node*> split(Node* n, int k) {
  if (!n) return {};
  push(n);
  if (cnt(n->l) >=
      k) {  // "n->val >= k" para usar lower_bound(k)
    auto [L, R] = split(n->l, k);
    n->l = R;
    n->recalc();
    return {L, n};
  } else {
    auto [L, R] =
        split(n->r, k - cnt(n->l) - 1);  // y solo "k"
    n->r = L;
    n->recalc();
    return {n, R};
  }
}

Node* merge(Node* l, Node* r) {
  push(l);
  push(r);
  if (!l) return r;
  if (!r) return l;
  if (l->y > r->y) {
    l->r = merge(l->r, r);
    return l->recalc(), l;
  } else {
    r->l = merge(l, r->l);
    return r->recalc(), r;
  }
}

Node* ins(Node* t, Node* n, int pos) {
  auto [l, r] = split(t, pos);
  return merge(merge(l, n), r);
}

void move(Node*& n, int l, int r, int k) {
  Node *a, *b, *c;
  tie(a, b) = split(n, l);
  tie(b, c) = split(b, r - l);
  if (k <= l)
    n = merge(ins(a, b, k), c);
  else
    n = merge(a, ins(c, b, k - r));
}

void shift(Node*& n, int l, int r) {
  if (l + 1 == r) return;
  Node *a, *b, *c, *d;
  tie(a, b) = split(n, l);
  tie(b, d) = split(b, r - l);
  tie(b, c) = split(b, r - l - 1);
  n = merge(merge(merge(a, c), b), d);
}

void reverse(Node*& n, int l, int r) {
  if (l + 1 == r) return;
  Node *a, *b, *c;
  tie(a, b) = split(n, l);
  tie(b, c) = split(b, r - l);
  apply(b);
  n = merge(merge(a, b), c);
}

int sum(Node* n, int l, int r) {
  Node *a, *b, *c;
  tie(a, b) = split(n, l);
  tie(b, c) = split(b, r - l);
  int ret = b->s;
  n = merge(merge(a, b), c);
  return ret;
}

int kth(Node* n, int k) {
  push(n);
  if (cnt(n->l) > k)
    return kth(n->l, k);
  else if (cnt(n->l) < k)
    return kth(n->r, k - cnt(n->l) - 1);
  return n->val;
}
```

3 NumberTheory

3.1 CRT

```
/*
 * Descripcion: Encuentra una solución x que satisfaga el
 * siguiente sistema de congruencias lineales:
 * x = a_1 (mod n_1)
 * . . .
 * x = a_k (mod n_k)
 * El Teorema Chino del Resto establece que para un
 * conjunto donde los modulos son coprimos entre pares,
 * existe una solucion unica modulo N, donde N es el
 * producto de las N_i A su vez, establece que para
 * modulos no coprimos, si para todo par de (i, j) se
 * cumple que a_i = a_j (mod g), entonces existe una
 * solucion unica modulo l (y si no, no existe), donde g =
 * gcd(m_1, ..., m_k) y l = lcm(m_1, ..., m_k) Tiempo en
 * general: O(k) Tiempo para 2 congruencias: O(log n)
 */

int crt(int a, int m, int b, int n) {
  if (n > m) swap(a, b), swap(m, n);
  int x, y, g = euclid(m, n, x, y);
  if ((a - b) % g) return NO_SOLUTION;
  x = (b - a) % n * x % n / g * m + a;
  return x < 0 ? x + m * n / g : x;
}

struct CRT {
  vector<pair<int, int>> equations;

  void clear() { equations.clear(); }

  void addEquation(int r, int m) {
    equations.push_back({r, m});
  }

  pair<int, int> solve() {
    if (equations.empty()) return {-1, -1};

    int m1 = equations[0].second,
        a1 = equations[0].first % m1;
    for (int i = 1; i < equations.size(); i++) {
      int m2 = equations[i].second,
          a2 = equations[i].first % m2;

      int g = gcd(m1, m2);
      if (a1 % g != a2 % g) return {-1, -1};

      int p, q;
      euclid(m1 / g, m2 / g, p, q);

      int mod = m1 / g * m2;
      mint<mod> x = mint<mod>(a1) * (m2 / g) * q +
                    mint<mod>(a2) * (m1 / g) * p;

      a1 = x.v, m1 = mod;
    }
    return {a1, m1};
  }
};
```

3.2 Discrete Log

```
/*
 * Descripcion: retorna el x mas chico tal que x > 0 y a^x
 *= b (mod m), o -1 si no existe tal x. discreteLog(a,1,m)
 *puede ser usado para calcular el orden de a.
 * Tiempo:
 *O(sqrt(m))
 *
 * Detalles: Este algoritmo utiliza el metodo baby-step
 *giant-step para encontrar (i,j) tal que a^(n * i) = b *
 *a^j (mod m), donde n > sqrt(m) y 0 < i, j <= n. Si a y m
 *son coprimos entonces a^j tiene inverso modular, lo que
 *significa que a^(i * n - j) = b (mod m).
 *
 * Sin embargo, esta implementación particular del
 *baby-step giant-step funciona incluso sin suponer que a
 *y m sean coprimos, usando la siguiente idea:
 *
 * Supongamos que p^x es un divisor primo de m. Entonces
 *tenemos 3 casos
 *       1. b es divisible por p^x
 *       2. b es divisible sólo por alguna p^y, 0<y<x
 *       3. b no es divisible por p
 * Lo importante a notar es que en el caso 2,
 *modLog(a,b,m) (si existe) no puede ser > sqrt(m)
 *(técnicamente no puede ser >= log2(m)). Así que, una vez
 *comprobados todos los exponentes de a que son <=
 *sqrt(m), el caso 2 ya no puede darse. El caso 2 es el
 *único caso delicado.
 *
 * Por tanto, la modificación que permite entrada no
 *coprima consiste en comprobar todos los exponentes de a
 *que sean <= n, y después manejar los casos no delicados
 *mediante una comprobación simple gcd(a^n,m) == gcd(b,m).
 */

int solve(int a, int b, int m) {
  int n = int(sqrt(m)) + 1, e = 1, f = 1, j = 1;
  unordered_map<ll, ll> A;
  while (j <= n && (e = f = e * a % m) != b % m)
    A[e * b % m] = j++;
  if (e == b % m) return j;
  if (gcd(m, e) == gcd(m, b))
    for (int i = 2; i <= n + 1; ++i)
      if (A.count(e = e * f % m)) return n * i - A[e];
  return -1;
}
```

3.3 Euclid

```
/**
 * Descripcion: Algoritmo extendido de Euclides,
 * retorna gcd(a, b) y encuentra dos enteros
 * (x, y) tal que ax + by = gcd(a, b), si solo
 * necesitas el gcd, utiliza __gcd (c++14 o
 * anteriores) o gcd (c++17 en adelante)
 * Si a y b son coprimos, entonces x es el
 * inverso de a mod b
 *
 * Tiempo: O(log n)
 */

int euclid(int a, int b, int &x, int &y) {
  if (!b) {
    x = 1, y = 0;
    return a;
  }
  int d = euclid(b, a % b, y, x);
  return y -= a / b * x, d;
}
```

3.4 Linear Diophantine

```
/**
 * Problema: Dado a, b y n. Encuentra x,y que
 * satisfagan la ecuacion ax + by = n.
 * Encuentra una de las soluciones, otras validas
 * pueden obtenerse por medio de:
 * x = x0 + k*(b/g), y = y0 - k*(a/g)
 *
 * Tiempo: O(log n)
 */
pair<int, int> linear_diophantine(int a, int b, int n) {
  int x0, y0, g = euclid(a, b, x0, y0);
  if (n % g) return NO_SOLUTION;
  x0 *= n / g, y0 *= n / g;
  return {x0, y0};
}
```

3.5 Mod Inverses

```
/**
 * Descripcion: Precalculo de modulos inversos hasta n.
 *
 * Tiempo: O(n)
 */
vector<mint> precalc_inverses(int n) {
  vector<mint> inv(n);
  inv[1] = 1;
  for (int i = 2; i < n; ++i) {
    inv[i] = inv[MOD % i] * (MOD / i) * -1;
  }
  return inv;
}
```

3.6 Modint

```
/**
 * Descripcion: Template para manejar enteros
 * bajo un modulo sin complicaciones
 * Para calcular el inverso, se asume que MOD
 * es primo (si no, usar gcd extendido)
 * Nota: Si se requiere instanciar para multiples MODs,
 * cambiar a `template<int MOD> struct mint { ... `
 * y en las inicializaciones usar `mi<MOD>` o `modint`.
 * Ejemplos de uso:
 *   mint res = a * b * c * d;
 *   res += mint(x) * x * a[j] * a[j] * (r - l);
 *
 * Tiempo: O(1)
 */
const int MOD = 998244353;

struct mint {
  int v;
  mint() : v(0) {}
  mint(ll _v) : v(int(_v % MOD)) { v += (v < 0) * MOD; }
  mint& operator+=(mint o) {
    if ((v += o.v) >= MOD) v -= MOD;
    return *this;
  }
  mint& operator-=(mint o) {
    if ((v -= o.v) < 0) v += MOD;
    return *this;
  }
  mint& operator*=(mint o) {
    v = int((ll)v * o.v % MOD);
    return *this;
  }
  friend mint be(mint a, ll p) {
    assert(p >= 0);
    return p == 0 ? 1
                  : be(a * a, p / 2) * (p & 1 ? a : 1);
  }
  friend mint inv(mint a) {
    assert(a.v != 0);
    return be(a, MOD - 2);
  }
  friend mint operator+(mint a, mint b) { return a += b; }
  friend mint operator-(mint a, mint b) { return a -= b; }
  friend mint operator*(mint a, mint b) { return a *= b; }
};
```

3.7 Multiplicative Functions

```
/**
 * Descripcion: algunas funciones multiplicativas.
 * Para arbitrarias, es bueno basarse en la criba lineal.
 *
 * Tiempo: O(N)
 */

void linear_sieve(int N) {
  vector<int> lp(N + 1), pr;
  for (int i = 2; i <= N; ++i) {
    if (lp[i] == 0) {
      lp[i] = i;
      pr.push_back(i);
    }
    for (int j = 0; i * pr[j] <= N; ++j) {
      lp[i * pr[j]] = pr[j];
      if (pr[j] == lp[i]) break;
    }
  }
}

void mobius(int N) {
  vector<int> mu(N), check(N);
  mu[1] = 1;
  int tot = 0;
  for (int i = 2; i < N; ++i) {
    if (!check[i]) {
      prime[tot++] = i;
      mu[i] = -1;
    }
    for (int j = 0; j < tot; ++j) {
      if (i * prime[j] > N) break;
      check[i * prime[j]] = true;
      if (i % prime[j] == 0) {
        mu[i * prime[j]] = 0;
        break;
      } else
        mu[i * prime[j]] = -mu[i];
    }
  }
}
```

3.8 Primitive Root

```
/*
 * Descripcion: en aritmetica modular, un numero g es
 * llamado una raiz primitiva modulo n si cada numero
 * coprimo con n es congruente a una potencia de g modulo
 * n. Matematicamente, esto es equivalente a decir que
 * para cada entero a tal que gcd(a, n) = 1, existe un
 * entero k tal que: g**k = a (mod n) Entonces, k es
 * llamado el indice o logaritmo discreto de a en la base
 * g modulo n. g tambien es llamado el generador del grupo
 * multiplicativo de enteros modulo n.
 * En particular, cuando n es primo, las potencias de la
 * raiz primitiva van por todos los numeros desde 1 hasta
 * n - 1 Una raiz primitiva modulo n existe si y solo si:
 *    n es 1, 2, 4 o
 *    n es una potencia de un primo impar (n = p**k) o
 *    n es 2 veces la potencia de un primo impar (n = 2 *
 * p**k) Nota. El codigo asume que el modulo p es un
 * primo. Para hacerlo funcionar para cualquier p, agregar
 * calculo de phi(p)
 * Tiempo: O(p * |factores de p| * log
 * p)
 */

int generator(int p) {
  vector<int> fact;
  int phi = p - 1, n = phi;
  for (int i = 2; i * i <= n; ++i) {
    if (n % i == 0) {
      fact.push_back(i);
      while (n % i == 0) {
        n /= i;
      }
    }
  }
  if (n > 1) {
    fact.push_back(n);
  }

  for (int res = 2; res <= p; ++res) {
    bool ok = true;
    for (int i = 0; i < fact.size() && ok; ++i) {
      ok &= be(res, phi / fact[i], p) != 1;
    }
    if (ok) {
      return res;
    }
  }
  return -1;
}
```

3.9 Sieve

```
/**
 * Descripcion: algoritmo para precalcular los
 * numeros primos menor o iguales a N.
 *
 * Tiempo: O(n log(log n))
 */
void sieve(int n) {
  vector<bool> is_prime(n + 1, 1);
  is_prime[0] = is_prime[1] = 0;
  for (ll p = 2; p <= n; p++) {
    if (!is_prime[p]) continue;
    for (ll i = p * p; i <= n; i += p) is_prime[i] = 0;
    primes.push_back(p);
  }
}
```

4 Combinatorial

4.1 Catalan

```
catalan = [0] * 105
catalan[0], catalan[1] = 1, 1
for i in range(2, n + 1):
    catalan[i] = 0
    for j in range(i):
        catalan[i] = catalan[i] + catalan[j] * catalan[i - j
            - 1]
```

4.2 Combinations

```
/**
 * Descripcion: calculo de combinaciones.
 * El primer metodo utiliza la formula convencional
 * y el segundo se  basa en el teorema de lucas, que
 * se usa con n grande y MOD chico.
 * Tiempo 1: O(n) de precalculo y O(1) por query.
 * Tiempo 2: O(m log_m(n))
 */

#include "ModInverses.h";

vector<mint> precalc_invfact(int n) {
  vector<mint> inv = precalc_inverses(n), invFact(n);
  invFact[0] = 1;
  for (int i = 1; i < n; i++)
    invfact[i] = invfact[i - 1] * inv[i];
}

mint comb(int n, int k) {
  return n < k ? 0
               : fact[n] * invfact[k] * invfact[n - k];
}

mint comb(int n, int k) {
  if (n < k || k < 0) return 0;
  if (n == k) return 1;
  return comb(n % MOD, k % MOD) * comb(n / MOD, k / MOD);
}
```

5 Numerical

5.1 Determinant

```
/*
 * Descripcion: Calcula la determinante de la matriz.
 * Nota: La matriz se modifica
 *
 * Tiempo: O(n^3)
 */

double det(vector<vector<double>>& a) {
  int n = SZ(a);
  double res = 1;
  for (int i = 0; i < n; ++i) {
    int b = i;
    for (int j = i + 1; j < n; ++j)
      if (abs(a[j][i]) > abs(a[b][i])) b = j;
    if (i != b) swap(a[i], a[b]), res *= -1;
    res *= a[i][i];
    if (res == 0) return 0;
    for (int j = i + 1; j < n; ++j) {
      double v = a[j][i] / a[i][i];
      if (v != 0)
        for (int k = i + 1; k < n; ++k)
          a[j][k] -= v * a[i][k];
    }
  }
  return res;
}

// Determinante con modulo.
// Sin el mod se obtiene la version de solo enteros.
mint detMod(vector<vector<mint>>& a) {
  int n = int(a.size());
  mint ans = 1;
  for (int i = 0; i < n; ++i) {
    for (int j = i + 1; j < n; ++j) {
      while (a[j][i] != 0) {
        ll t = a[i][i] / a[j][i];
        if (t)
          for (int k = i; k < n; ++k)
            a[i][k] -= a[j][k] * t;
        swap(a[i], a[j]);
        ans *= -1;
      }
    }
    ans *= a[i][i];
    if (!ans.v) return 0;
  }
  return ans;
}
```

5.2 FFT

```
/*
 * Descripcion: Este algoritmo permite multiplicar dos
 * polinomios de longitud n
 * Tiempo: O(n log n)
 */

typedef double ld;
const ld PI = acos(-1.0L);
const ld one = 1;

typedef complex<ld> C;
typedef vector<ld> vd;

void fft(vector<C> &a) {
  int n = SZ(a), L = 31 - __builtin_clz(n);
  static vector<complex<ld>> R(2, 1);
  static vector<C> rt(2, 1);  // (^ 10% faster if double)
  for (static int k = 2; k < n; k *= 2) {
    R.resize(n);
    rt.resize(n);
    auto x = polar(one, PI / k);
    FOR(i, k, 2 * k)
    rt[i] = R[i] = i & 1 ? R[i / 2] * x : R[i / 2];
  }
  vi rev(n);
  FOR(i, 0, n)
  rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
  FOR(i, 0, n)
  if (i < rev[i]) swap(a[i], a[rev[i]]);
  for (int k = 1; k < n; k *= 2)
    for (int i = 0; i < n; i += 2 * k) FOR(j, 0, k) {
        // C z = rt[j+k] * a[i+j+k]; // (25% faster if
        // hand-rolled)  /// include-line
        auto x = (ld *)&rt[j + k],
             y = (ld *)&a[i + j + k];  /// exclude-line
        C z(x[0] * y[0] - x[1] * y[1],
            x[0] * y[1] + x[1] * y[0]);  /// exclude-line
        a[i + j + k] = a[i + j] - z;
        a[i + j] += z;
      }
}

typedef vector<ll> vl;

vl conv(const vl &a, const vl &b) {
  if (a.empty() || b.empty()) return {};
  vl res(SZ(a) + SZ(b) - 1);
  int L = 32 - __builtin_clz(SZ(res)), n = 1 << L;
  vector<C> in(n), out(n);
  copy(all(a), begin(in));
  FOR(i, 0, SZ(b))
  in[i].imag(b[i]);
  fft(in);
  for (C &x : in) x *= x;
  FOR(i, 0, n)
  out[i] = in[-i & (n - 1)] - conj(in[i]);
  fft(out);
  FOR(i, 0, SZ(res))
  res[i] = floor(imag(out[i]) / (4 * n) + 0.5);
  return res;
}

vl convMod(const vl &a, const vl &b, const int &M) {
  if (a.empty() || b.empty()) return {};
  vl res(SZ(a) + SZ(b) - 1);
  int B = 32 - __builtin_clz(SZ(res)), n = 1 << B,
      cut = int(sqrt(M));
  vector<C> L(n), R(n), outs(n), outl(n);
  FOR(i, 0, SZ(a))
  L[i] = C((int)a[i] / cut, (int)a[i] % cut);
  FOR(i, 0, SZ(b))
  R[i] = C((int)b[i] / cut, (int)b[i] % cut);
  fft(L), fft(R);
  FOR(i, 0, n) {
    int j = -i & (n - 1);
    outl[j] = (L[i] + conj(L[j])) * R[i] / (2.0 * n);
    outs[j] = (L[i] - conj(L[j])) * R[i] / (2.0 * n) / 1i;
  }
  fft(outl), fft(outs);
  FOR(i, 0, SZ(res)) {
    ll av = ll(real(outl[i]) + .5),
       cv = ll(imag(outs[i]) + .5);
    ll bv =
        ll(imag(outl[i]) + .5) + ll(real(outs[i]) + .5);
    res[i] = ((av % M * cut + bv) % M * cut + cv) % M;
  }
  return res;
}
```

5.3 Gauss

```
/*
 * Descripcion: resolvedor de sistemas de ecuaciones
 * lineales. a[i] = coeficientes de la i-esima ecuacion,
 * a[i][m] es su suma. Retorna el numero de soluciones.
 * Recibe como referencia el vector solucion. Encuentra
 * cualquiera. El primer metodo funciona igual sobre
 * modulo, sin necesitar las heuristicas. Usar mint. Caso
 * especial: si se trabaja modulo 2, A - B = A xor B. Por
 * lo tanto, se puede usar bitset para optimizar el
 * pivoteo a O(n^3/64)
 * Tiempo: O(n^3)
 */

int gauss(vector<vector<double>> &a,
          vector<double> &ans) {
  int n = int(a.size()), m = int(a[0].size()) - 1;
  vector<int> where(m, -1);
  for (int col = 0, row = 0; col < m && row < n; ++col) {
    int sel = row;
    for (int i = row; i < n; ++i)
      if (abs(a[i][col]) > abs(a[sel][col])) sel = i;
    if (abs(a[sel][col]) < EPS) continue;

    for (int i = col; i <= m; ++i)
      swap(a[sel][i], a[row][i]);
    where[col] = row;

    for (int i = 0; i < n; ++i)
      if (i != row) {
        double c = a[i][col] / a[row][col];
        for (int j = col; j <= m; ++j)
          a[i][j] -= a[row][j] * c;
      }
    ++row;
  }

  ans.assign(m, 0);
  for (int i = 0; i < m; ++i)
    if (where[i] != -1)
      ans[i] = a[where[i]][m] / a[where[i]][i];
  for (int i = 0; i < n; ++i) {
    double sum = 0;
    for (int j = 0; j < m; ++j) sum += ans[j] * a[i][j];
    if (abs(sum - a[i][m]) > EPS) return 0;
  }

  for (int i = 0; i < m; ++i)
    if (where[i] == -1) return INF;

  return 1;
}
```

5.4 Simplex

```
/*
 * Descripcion: Es un algoritmo de programacion lineal
 * para resolver problemas de optimizacion. El objetivo es
 * maximizar una funcion lineal (c) sujeta a un conjunto
 * de restricciones lineales (b). Maximiza cTx sujeto a Ax
 * < b, x > 0
 * Uso: T val = LPSolver(A,b,c).solve(x);
 * Entrada: A -> MxN matriz de r
 *          b -> un vector de tamano M (restricciones)
 *          c -> un vector de tamano N (maximizar)
 *          x -> un vector donde la solucion optima es
 * almacenado Salida: el valor maximo de la solucion
 * optima. (Infinito en un caso ilimtado)
 * Tiempo:
 * O(NM*#pivotes) donde el #pivotes en el peor caso es
 * exponencial
 */

typedef double T;
typedef vector<T> vd;
typedef vector<vd> vvd;

const T eps = 1e-8, inf = 1 / .0;
#define MP make_pair
```

```
#define ltj(X) \
  if (s == -1 || MP(X[j], N[j]) < MP(X[s], N[s])) s = j

struct LPSolver {
  int m, n;
  vi N, B;
  vvd D;

  LPSolver(const vvd& A, const vd& b, const vd& c)
      : m(SZ(b)),
        n(SZ(c)),
        N(n + 1),
        B(m),
        D(m + 2, vd(n + 2)) {
    FOR(i, 0, m)
    FOR(j, 0, n)
    D[i][j] = A[i][j];
    FOR(i, 0, m) {
      B[i] = n + i;
      D[i][n] = -1;
      D[i][n + 1] = b[i];
    }
    FOR(j, 0, n) {
      N[j] = j;
      D[m][j] = -c[j];
    }
    N[n] = -1;
    D[m + 1][n] = 1;
  }

  void pivot(int r, int s) {
    T *a = D[r].data(), inv = 1 / a[s];
    FOR(i, 0, m + 2)
    if (i != r && abs(D[i][s]) > eps) {
      T *b = D[i].data(), inv2 = b[s] * inv;
      FOR(j, 0, n + 2)
      b[j] -= a[j] * inv2;
      b[s] = a[s] * inv2;
    }
    FOR(j, 0, n + 2)
    if (j != s) D[r][j] *= inv;
    FOR(i, 0, m + 2)
    if (i != r) D[i][s] *= -inv;
    D[r][s] = inv;
    swap(B[r], N[s]);
  }

  bool simplex(int phase) {
    int x = m + phase - 1;
    for (;;) {
      int s = -1;
      FOR(j, 0, n + 1)
      if (N[j] != -phase) ltj(D[x]);
      if (D[x][s] >= -eps) return true;
      int r = -1;
      FOR(i, 0, m) {
        if (D[i][s] <= eps) continue;
        if (r == -1 ||
            MP(D[i][n + 1] / D[i][s], B[i]) <
                MP(D[r][n + 1] / D[r][s], B[r]))
          r = i;
      }
      if (r == -1) return false;
      pivot(r, s);
    }
  }

  T solve(vd& x) {
    int r = 0;
    FOR(i, 1, m)
    if (D[i][n + 1] < D[r][n + 1]) r = i;
    if (D[r][n + 1] < -eps) {
      pivot(r, n);
      if (!simplex(2) || D[m + 1][n + 1] < -eps)
        return -inf;
      FOR(i, 0, m)
      if (B[i] == -1) {
        int s = 0;
        FOR(j, 1, n + 1)
          ltj(D[i]);
        pivot(i, s);
      }
    }
    bool ok = simplex(1);
    x = vd(n);
    FOR(i, 0, m)
    if (B[i] < n) x[B[i]] = D[i][n + 1];
    return ok ? D[m][n + 1] : inf;
  }
};
```

5.5 Simpson

```
/*
 * Descripcion: Calcula el valor de una integral definida
 * N es el numero de pasos. Mas pasos implica mas
 * precision
 * Tiempo: O(pasos)
 */

const int N = 1000 * 1000;
double simpson_integration(double a, double b) {
  double h = (b - a) / N;
  double s = f(a) + f(b);
  for (int i = 1; i <= N - 1; ++i) {
    double x = a + h * i;
    s += f(x) * ((i & 1) ? 4 : 2);
  }
  s *= h / 3;
  return s;
}
```

5.6 Xor Basis

```
/*
 * Descripcion: calcula el XOR basis en Row Echelon Form.
 * El K-esimo menor elemento (0-indexed) es el mapeo
 * binario de K a los elementos del basis.
 * Si no se requiere getKth(), se puede omitir el segundo
 * for dentro de insert() y la insercion lineal,
 * cambiandolo simplemente por un `basis.pb(x)`. Los
 * elementos se mantienen ordenados descendentemente.
 *
 * Tiempo: O(D), donde D es el numero de bits del maximo
 * valor.
 */
struct XorBasis {
  vector<int> basis;

  void insert(int x) {
    for (int b : basis) x = min(x, x ^ b);
    if (!x) return;

    for (int& b : basis) b = min(b, b ^ x);
    basis.insert(lower_bound(ALL(basis), x, greater<>()),
                 x);
  }

  int getKth(int k) {
    int x = 0;
    for (int i = 0; i < SZ(basis); i++) {
      if ((k >> (SZ(basis) - 1 - i)) & 1) {
        x ^= basis[i];
      }
    }
    return x;
  }
};
```

6 Graphs

6.1 2SAT

```cpp
/**
 * Descripcion: estructura para resolver el problema de
 * TwoSat: dadas disyunciones del tipo (a or b) donde las
 * variables pueden o no estar negadas, se necesita saber
 * si es posible asignarle un valor a cada variable de tal
 * modo que cada disyuncion se cumpla. Las variables
 * negadas son representadas por inversiones de bits (~x)
 *
 * Uso:
 * TwoSat ts(numero de variables booleanas);
 * ts.either(0, ~3);           La variable 0 es verdadera
 * o la variable 3 es falsa ts.setValue(2);             La
 * variable 2 es verdadera ts.atMostOne({0, ~1, 2});   <=
 * 1 de vars 0, ~1 y 2 son verdedero ts.solve(); Retorna
 * verdadero si existe solucion ts.values[0..N-1] Tiene
 * los valores asignados a las variables
 * Tiempo: O(N + E),
 * donde N es el numero de variables booleanas y E es el
 * numero de clausulas
 */

struct TwoSat {
  int N;
  vector<vi> g;
  vi values;  // 0 = false, 1 = true

  TwoSat(int n = 0) : N(n), g(2 * n) {}

  int addVar() {
    g.emplace_back();
    g.emplace_back();
    return N++;
  }

  // Agregar una disyuncion
  void either(
      int x,
      int y) {  // Nota: (a v b), es equivalente a la
                // expresion (~a -> b) n (~b -> a)
    x = max(2 * x, -1 - 2 * x),
    y = max(2 * y, -1 - 2 * y);
    g[x].push_back(y ^ 1), g[y].push_back(x ^ 1);
  }
  void setValue(int x) { either(x, x); }
  void implies(int x, int y) { either(~x, y); }
  void make_diff(int x, int y) {
    either(x, y);
    either(~x, ~y);
  }
  void make_eq(int x, int y) {
    either(~x, y);
    either(x, ~y);
  }

  void atMostOne(const vi& li) {
    if (li.size() <= 1) return;
    int cur = ~li[0];
    for (int i = 2; i < li.size(); i++) {
      int next = addVar();
      either(cur, ~li[i]);
      either(cur, next);
      either(~li[i], next);
      cur = ~next;
    }
    either(cur, ~li[1]);
  }

  vi dfs_num, comp;
  stack<int> st;
  int time = 0;
  int tarjan(int u) {
    int x, low = dfs_num[u] = ++time;
    st.push(u);
    for (int v : g[u])
      if (!comp[v])
        low = min(low, dfs_num[v] ?: tarjan(v));
    if (low == dfs_num[u]) {
      do {
        x = st.top();
        st.pop();
        comp[x] = low;
        if (values[x >> 1] == -1) values[x >> 1] = x & 1;
      } while (x != u);
    }
    return dfs_num[u] = low;
  }

  bool solve() {
    values.assign(N, -1);
    dfs_num.assign(2 * N, 0);
    comp.assign(2 * N, 0);
    for (int i = 0; i < 2 * N; i++)
      if (!comp[i]) tarjan(i);
    for (int i = 0; i < N; i++)
      if (comp[2 * i] == comp[2 * i + 1]) return 0;
    return 1;
  }
};
```

6.2 Biconnected Components

```cpp
/**
 * Descripcion: Encuentra todos los componentes
 * biconectados en un grafo no dirigido y ejecuta un
 * callback f para las aristas de cada uno. Una arista que
 * no pertenece a ningun componente es un puente. Soporta
 * multiaristas usando IDs unicos por arista.
 * Uso:
 * vector<vector<pi>> ed(n);
 * // para cada arista i entre (u, v): ed[u].pb({v, i});
 * ed[v].pb({u, i}); bicomps(ed, [&](const vi& edge_list)
 * { ... });
 * Tiempo: O(V + E)
 */

template <class F>
void bicomps(vector<vector<pi>>& ed, F f) {
  int n = SZ(ed), Time = 0;
  vi num(n), st;
  auto dfs = [&](auto& self, int at, int par) -> int {
    int me = num[at] = ++Time, top = me;
    for (auto [y, e] : ed[at])
      if (e != par) {
        if (num[y]) {
          top = min(top, num[y]);
          if (num[y] < me) st.pb(e);
        } else {
          int si = SZ(st);
          int up = self(self, y, e);
          top = min(top, up);
          if (up == me) {
            st.pb(e);
            f(vi(begin(st) + si, end(st)));
            st.resize(si);
          } else if (up < me) {
            st.pb(e);
          }
        }
      }
    return top;
  };
  FOR(i, 0, n) if (!num[i]) dfs(dfs, i, -1);
}
```

6.3 Bridges And Articulation Points

```cpp
/**
 * Descripcion: basado en la clasificacion de aristas por
 * medio del arbol de dfs, donde una arista es puente si
 * no existe un back-edge que cruce por encima, los puntos
 * de articulacion se obtienen de forma similar. Retorna
 * los puentes y un vector donde art[i] = 1 si el i-esimo
 * nodo es un punto de articulacion.
 * Tiempo: O(V + E)
 */

pair<vector<pi>, vi> findBridgesAndArticulationPoints(
    vector<vi>& g) {
  int n = SZ(g), t = 0;
  vector<pi> bridges;
  vi tin(n, -1), low(n, -1), art(n);
  auto dfs = [&](auto self, int u, int p = -1) -> void {
    tin[u] = low[u] = t++;
    int children = 0;
    for (int v : g[u])
      if (v != p) {
        if (tin[v] != -1) {
          low[u] = min(low[u], tin[v]);
          continue;
        }
        self(self, v, u);
        if (low[v] >= tin[u] && p != -1) art[u] = 1;
        if (low[v] > tin[u]) bridges.pb({u, v});
        low[u] = min(low[u], low[v]);
        children++;
      }
    if (p == -1 && children > 1) art[u] = 1;
  };
  FOR(u, 0, n) if (tin[u] == -1) dfs(dfs, u);
  return {bridges, art};
}
```

6.4 Dominator Tree

```cpp
/**
 * Descripcion: Construye el arbol de dominadores para un
 * grafo dirigido dado un nodo raiz. Un nodo 'u' domina a
 * un nodo 'v' si todos los caminos desde la raiz hasta
 * 'v' deben pasar obligatoriamente por 'u'. El dominador
 * inmediato de un nodo 'v' es el unico nodo 'u' que
 * domina a 'v' y no domina a ningun otro dominador de
 * 'v', siendo asi su padre directo en el arbol.
 * Uso: vi
 * iDom = getDominatorTree(adj, root); donde iDom[u] es el
 * dominador inmediato del nodo u
 * Tiempo: O((V + E) log V)
 */

vi getDominatorTree(const vector<vi>& adj, int r) {
  int n = SZ(adj), t = 0;
  vi id(n, -1), rId(n), relabeledP(n, -1);
  vector<vi> rAdj(n);
  auto eulerTour = [&](auto&& self, int u) -> void {
    id[u] = t++;
    rId[id[u]] = u;
    for (int v : adj[u]) {
      if (id[v] == -1) {
        relabeledP[v] = id[u];
        self(self, v);
      }
      rAdj[v].pb(u);
    }
  };
  eulerTour(eulerTour, r);

  vi dsuP(t);
  vector<pi> dsuW(t, {INT_MAX, -1});
  iota(dsuP.begin(), dsuP.end(), 0);
  auto query = [&](auto&& self, int u) -> pi {
    if (dsuP[u] == u) return dsuW[u];
    dsuW[u] = min(dsuW[u], self(self, dsuP[u]));
    dsuP[u] = dsuP[dsuP[u]];
```

```
    return dsuW[u];
  };

  vi sDom(t, INT_MAX), iDom(n, -1);
  vector<vi> pending(t);
  ROF(i, t, 0) {
    for (int j : pending[i]) {
      auto [mn, k] = query(query, j);
      iDom[rId[j]] =
          mn == sDom[j] ? rId[sDom[j]] : rId[k];
    }
    if (!i) break;
    int u = rId[i];
    for (int v : rAdj[u]) {
      int j = id[v];
      sDom[i] =
          min(sDom[i], j < i ? j : query(query, j).first);
    }
    dsuW[i] = {sDom[i], i};
    dsuP[i] = relabeledP[u];
    pending[sDom[i]].pb(i);
  }
  FOR(i, 1, t) {
    int u = rId[i];
    if (iDom[u] != rId[sDom[i]]) iDom[u] = iDom[iDom[u]];
  }
  return iDom;
}
```

6.5 Maximal Cliques

```
/**
 * Descripcion: Corre un callback para todos los cliques
 * maximales en un grafo (dado como una matriz simetrica
 * de bitset; aristas propias no permitidas). El callback
 * recibe un bitset representando al clique maximal.
 *
 * Tiempo: O(3^{V/3}), mucho mas veloz para grafos
 * dispersos.
 */
/// Posible optimizacion: en el mayor nivel de recursion
/// ignora 'cands' y muevete hacia nodos en orden de grado
/// creciente, donde los los grados bajan al eliminar
/// nodos (en su mayoria irrelevante dado MaximumClique)

typedef bitset<128> B;
template <class F>
void cliques(vector<B>& eds, F f, B P = ~B(), B X = {},
             B R = {}) {
  if (!P.any()) {
    if (!X.any()) f(R);
    return;
  }
  auto q = (P | X)._Find_first();
  auto cands = P & ~eds[q];
  rep(i, 0, sz(eds)) if (cands[i]) {
    R[i] = 1;
    cliques(eds, f, P & eds[i], X & eds[i], R);
    R[i] = P[i] = 0;
    X[i] = 1;
  }
}
```

6.6 Maximum Clique

```
/**
 * Descripcion: Rapidamente encuentra el clique maximo de
 * un grafo (dado como una matriz simetrica de bitset;
 * aristas propias no permitidas). Puede ser usado para
 * encontrar un conjunto independiente maximo, encontrando
 * un clique del grafo complemento.
 *
 * Tiempo: aproximadamente 1s para n=155 y el peor caso
 * para grafos aleatorios (p=.90), mas veloz para grafos
 * dispersos.
 */
typedef vector<bitset<200>> vb;
struct Maxclique {
  double limit = 0.025, pk = 0;
  struct Vertex {
    int i, d = 0;
  };
  typedef vector<Vertex> vv;
  vb e;
  vv V;
  vector<vi> C;
  vi qmax, q, S, old;
  void init(vv& r) {
    for (auto& v : r) v.d = 0;
    for (auto& v : r)
      for (auto j : r) v.d += e[v.i][j.i];
    sort(ALL(r),
         [](auto a, auto b) { return a.d > b.d; });
    int mxD = r[0].d;
    FOR(i, 0, sz(r)) r[i].d = min(i, mxD) + 1;
  }
  void expand(vv& R, int lev = 1) {
    S[lev] += S[lev - 1] - old[lev];
    old[lev] = S[lev - 1];
    while (sz(R)) {
      if (sz(q) + R.back().d <= sz(qmax)) return;
      q.push_back(R.back().i);
      vv T;
      for (auto v : R)
        if (e[R.back().i][v.i]) T.push_back({v.i});
      if (sz(T)) {
        if (S[lev]++ / ++pk < limit) init(T);
        int j = 0, mxk = 1,
            mnk = max(sz(qmax) - sz(q) + 1, 1);
        C[1].clear(), C[2].clear();
        for (auto v : T) {
          int k = 1;
          auto f = [&](int i) { return e[v.i][i]; };
          while (any_of(ALL(C[k]), f)) k++;
          if (k > mxk) mxk = k, C[mxk + 1].clear();
          if (k < mnk) T[j++].i = v.i;
          C[k].push_back(v.i);
        }
        if (j > 0) T[j - 1].d = 0;
        FOR(k, mnk, mxk + 1)
        for (int i : C[k]) T[j].i = i,
                               T[j++].d = k;
        expand(T, lev + 1);
      } else if (sz(q) > sz(qmax))
        qmax = q;
      q.pop_back(), R.pop_back();
    }
  }
  vi maxClique() {
    init(V), expand(V);
    return qmax;
  }
  Maxclique(vb conn)
      : e(conn), C(sz(e) + 1), S(sz(C)), old(S) {
    FOR(i, 0, sz(e)) V.push_back({i});
  }
};
```

6.7 SCC

```
/**
 * Descripcion: algoritmo de Tarjan para la busqueda de
 * componentes fuertemente conexos (SCC)
 * Idea: Los SCC forman subarboles en el arbol de
 * expansion de la DFS. Ademas de calcular tin(u) y low(u)
 * para cada vertice, se agrega el vertice u al final de
 * una pila z y mantenemos la informacion de que vertices
 * estan siendo explorados, mediante vi vis. Solo los
 * vertices visitados (parte del SCC actual) pueden
 * actualizar low(u). Si low(u) = tin(u), u es la raiz de
 * un SCC y sus miembros pueden obtenerse sacando de la
 * pila. Retorna un vector, donde scc[u] es el indice de
 * su SCC.
 * Tiempo: O(V + E)
 */
vi scc(vector<vi>& g) {
  int n = SZ(g), t = 0, ncomps = 0;
  vi tin(n), scc(n, -1), z;
  auto dfs = [&](auto&& self, int u) -> int {
    int low = tin[u] = ++t, x;
    z.push_back(u);
    for (auto v : g[u])
      if (scc[v] < 0)
        low = min(low, tin[v] ?: self(self, v));
    if (low == tin[u]) {
      do {
        x = z.back();
        z.pop_back();
        scc[x] = ncomps;
      } while (x != u);
      ncomps++;
    }
    return tin[u] = low;
  };
  FOR(i, 0, n) if (scc[i] == -1) dfs(dfs, i);
  return scc;
}
```

6.8 General Matching

```
/**
 * Descripcion: Variante de la implementacion de Gabow
 * para el algoritmo de Edmonds-Blossom. Maximo
 * emparejamiento sin peso para un grafo en general, con
 * indexacion. Si despues de terminar la llamada a
 * solve(), white[v] = 0, v es parte de cada matching
 * maximo.
 * Tiempo: O(VE), mas rapido en la practica.
 */
struct MaxMatching {
  int N;
  vector<vi> adj;
  vector<pi> label;
  vi mate, first;
  vector<bool> white;

  MaxMatching(int _N)
      : N(_N),
        adj(vector<vi>(N + 1)),
        mate(vi(N + 1)),
        first(vi(N + 1)),
        label(vector<pi>(N + 1)),
        white(vector<bool>(N + 1)) {}

  void addEdge(int u, int v) {
    adj.at(u).pb(v), adj.at(v).pb(u);
  }

  int group(int x) {
    if (white[first[x]]) first[x] = group(first[x]);
    return first[x];
  }
  void match(int p, int b) {
    swap(b, mate[p]);
    if (mate[b] != p) return;
    if (!label[p].second)
      mate[b] = label[p].first,
      match(label[p].first, b);  // label del vertice
    else
      match(label[p].first, label[p].second),
          match(label[p].second,
                label[p].first);  // label de la arista
  }
```

```
  bool augment(int st) {
    assert(st);
    white[st] = 1;
    first[st] = 0;
    label[st] = {0, 0};

    queue<int> q;
    q.push(st);

    while (!q.empty()) {
      int a = q.front();
      q.pop();  // vertice exterior
      for (auto& b : adj[a]) {
        assert(b);
        if (white[b]) {
          int x = group(a), y = group(b), lca = 0;
          while (x || y) {
            if (y) swap(x, y);
            if (label[x] == pi{a, b}) {
              lca = x;
              break;
            }
            label[x] = {a, b};
            x = group(label[mate[x]].first);
          }
          for (int v : {group(a), group(b)})
            while (v != lca) {
              assert(!white[v]);  // haz blanco a todo a
                                  // lo largo del camino
              q.push(v);
              white[v] = true;
              first[v] = lca;
              v = group(label[mate[v]].first);
            }
        } else if (!mate[b]) {
          mate[b] = a;
          match(a, b);
          white = vector<bool>(N + 1);  // reset
          return true;
        } else if (!white[mate[b]]) {
          white[mate[b]] = true;
          first[mate[b]] = b;
          label[b] = {0, 0};
          label[mate[b]] = pi{a, 0};
          q.push(mate[b]);
        }
      }
    }
    return false;
  }
  int solve() {
    int ans = 0;
    FOR(st, 1, N + 1) if (!mate[st]) ans += augment(st);
    FOR(st, 1, N + 1)
    if (!mate[st] && !white[st]) assert(!augment(st));
    return ans;
  }
};
```

6.9 Hopcroft Karp

```
/**
 * Descripcion: Algoritmo de Hopcroft-Karp para maximo
 * emparejamiento bipartito. El grafo g contiene los
 * vecinos de la particion izquierda. r debe inicializarse
 * con tamano m (particion derecha) lleno de -1. Retorna
 * el tamano del matching y r[v] almacena el nodo
 * izquierdo asignado (-1 si ninguno).
 * Uso: vi r(m, -1);
 * int matching = hopcroftKarp(g, r);
 *
 * Tiempo: O(E\sqrt{V})
 */

int hopcroftKarp(vector<vi>& g, vi& r) {
  int n = SZ(g), res = 0;
  vi l(n, -1), q(n), d(n);
  auto dfs = [&](auto f, int u) -> bool {
    int t = exchange(d[u], 0) + 1;
    for (int v : g[u])
      if (r[v] == -1 || (d[r[v]] == t && f(f, r[v]))) {
        return l[u] = v, r[v] = u, 1;
    return 0;
  };
  for (int t = 0, f = 0;; t = f = 0, d.assign(n, 0)) {
    FOR(i, 0, n) if (l[i] == -1) q[t++] = i, d[i] = 1;
    FOR(i, 0, t) for (int v : g[q[i]]) {
      if (r[v] == -1)
        f = 1;
      else if (!d[r[v]])
        d[r[v]] = d[q[i]] + 1, q[t++] = r[v];
    }
    if (!f) return res;
    FOR(i, 0, n) if (l[i] == -1) res += dfs(dfs, i);
  }
}
```

6.10 Hungarian

```
/**
 * Descripcion: Dado un grafo bipartito ponderado,
 * empareja cada nodo en la izquierda con un nodo en la
 * derecha, tal que ningun nodo pertenece a 2
 * emparejamientos y que la suma de los pesos de las
 * aristas usadas es minima. Toma a[N][M], donde a[i][j]
 * es el costo de emparejar L[i] con R[j], retorna (costo
 * minimo, match), donde L[i] es emparejado con
 * R[match[i]], negar costos si se requiere el
 * emparejamiento maximo, se requiere que N <= M.
 * Tiempo:
 * O(N^2 M)
 */
template <typename T>
pair<T, vi> hungarian(const vector<vector<T>> &a) {
#define INF numeric_limits<T>::max()
  if (a.empty()) return {0, {}};
  int n = SZ(a) + 1, m = SZ(a[0]) + 1;
  vi p(m), ans(n - 1);
  vector<T> u(n), v(m);
  FOR(i, 1, n) {
    p[0] = i;
    int j0 = 0;  // agregar trabajador "dummy" 0
    vector<T> dist(m, INF);
    vi pre(m, -1);
    vector<bool> done(m + 1);
    do {  // dijkstra
      done[j0] = true;
      int i0 = p[j0], j1;
      T delta = INF;
      FOR(j, 1, m)
      if (!done[j]) {
        auto cur = a[i0 - 1][j - 1] - u[i0] - v[j];
        if (cur < dist[j]) dist[j] = cur, pre[j] = j0;
        if (dist[j] < delta) delta = dist[j], j1 = j;
      }
      FOR(j, 0, m) {
        if (done[j])
          u[p[j]] += delta, v[j] -= delta;
        else
          dist[j] -= delta;
      }
      j0 = j1;
    } while (p[j0]);
    while (j0) {  // actualizar camino alternativo
      int j1 = pre[j0];
      p[j0] = p[j1], j0 = j1;
    }
  }
  FOR(j, 1, m)
  if (p[j]) ans[p[j] - 1] = j - 1;
  return {-v[0], ans};
}
```

6.11 Kuhn

```
/**
 * Descripcion: Algoritmo simple para maximo
 * emparejamiento bipartito. el grafo g debe de ser una
 * lista de los vecinos de la particion izquierda y m el
 * numero de nodos en la particion derecha. Retorna
 * (|matching|, btoa) donde btoa[i] sera el emparejamiento
 * para el vertice i del lado derecho o -1 si no lo tiene
 *
 * Tiempo: O(VE)
 */
pair<int, vi> kuhn(vector<vi>& g, int m) {
  vi vis, btoa(m, -1);
  auto dfs = [&](auto self, int j) -> bool {
    if (btoa[j] == -1) return 1;
    vis[j] = 1;
    int di = btoa[j];
    for (int e : g[di])
      if (!vis[e] && self(self, e)) {
        btoa[e] = di;
        return 1;
      }
    return 0;
  };
  FOR(i, 0, SZ(g)) {
    vis.assign(SZ(btoa), 0);
    for (int j : g[i])
      if (dfs(dfs, j)) {
        btoa[j] = i;
        break;
      }
  }
  return {SZ(btoa) - (int)count(ALL(btoa), -1), btoa};
}
```

6.12 Minimum Vertex Cover

```
/**
 * Descripcion: Encuentra los vertices en el minimum
 * vertex cover de un grafo bipartito, si solo se quiere
 * la cardinalidad leer teoria.
 * Tiempo: depende el
 * algoritmo de matching que se use
 */

#include "Kuhn.h"  // o hopcroft karp

vi cover(vector<vi>& g, int n, int m) {
  auto [res, btoa] = kuhn(g, m);
  vector<bool> lfound(n, true), seen(m);
  for (int it : btoa)
    if (it != -1) lfound[it] = false;
  vi q, cover;
  FOR(i, 0, n) if (lfound[i]) q.push_back(i);
  while (!q.empty()) {
    int u = q.back();
    q.pop_back();
    lfound[u] = 1;
    for (int v : g[u])
      if (!seen[v] && btoa[v] != -1) {
        seen[v] = true;
        q.push_back(btoa[v]);
      }
  }
  FOR(i, 0, n) if (!lfound[i]) cover.push_back(i);
  FOR(i, 0, m) if (seen[i]) cover.push_back(n + i);
  assert(sz(cover) == res);
  return cover;
}
```

6.13 Directed MST

```
/**
 * Descripcion: encuentra un arbol minimo de expansion de
 * un grafo dirigido, dado un nodo raiz. Si no existe
 * ningun MST, se retorna -1.
 * Tiempo: O(E \log V) Estatus:
 * testeado en NWERC 2018 fastestspeedrun
 */

struct Edge {
  int a, b;
  ll w;
};
struct Node {  /// lazy skew heap node
  Edge key;
  Node *l, *r;
  ll delta;
  void prop() {
    key.w += delta;
    if (l) l->delta += delta;
    if (r) r->delta += delta;
    delta = 0;
  }
  Edge top() {
    prop();
    return key;
  }
};
Node* merge(Node* a, Node* b) {
  if (!a || !b) return a ?: b;
  a->prop(), b->prop();
  if (a->key.w > b->key.w) swap(a, b);
  swap(a->l, (a->r = merge(b, a->r)));
  return a;
}
void pop(Node*& a) {
  a->prop();
  a = merge(a->l, a->r);
}

pair<ll, vi> dmst(int n, int r, vector<Edge>& g) {
  RollbackUF uf(n);
  vector<Node*> heap(n);
  for (Edge e : g)
    heap[e.b] = merge(heap[e.b], new Node{e});
  ll res = 0;
  vi seen(n, -1), path(n), par(n);
  seen[r] = r;
  vector<Edge> Q(n), in(n, {-1, -1}), comp;
  deque<tuple<int, int, vector<Edge>>> cycs;
  for (int s = 0; s < n; ++s) {
    int u = s, qi = 0, w;
    while (seen[u] < 0) {
      if (!heap[u]) return {-1, {}};
      Edge e = heap[u]->top();
      heap[u]->delta -= e.w, pop(heap[u]);
      Q[qi] = e, path[qi++] = u, seen[u] = s;
      res += e.w, u = uf.find(e.a);
      if (seen[u] == s) {  /// found cycle, contract
        Node* cyc = 0;
        int end = qi, time = uf.time();
        do cyc = merge(cyc, heap[w = path[--qi]]);
        while (uf.join(u, w));
        u = uf.find(u), heap[u] = cyc, seen[u] = -1;
        cycs.push_front({u, time, {&Q[qi], &Q[end]}});
      }
    }
    for (int i = 0; i < qi; ++i)
      in[uf.find(Q[i].b)] = Q[i];
  }

  for (auto& [u, t, comp] :
       cycs) {  // restore sol (optional)
    uf.rollback(t);
    Edge inEdge = in[u];
    for (auto& e : comp) in[uf.find(e.b)] = e;
    in[uf.find(inEdge.b)] = inEdge;
  }
  for (int i = 0; i < n; ++i) par[i] = in[i].a;
  return {res, par};
}
```

6.14 Kruskal

```
/**
 * Descripcion: tiene como principal funcion calcular la
 * suma del peso de las aristas del arbol minimo de
 * expansion (MST) de un grafo no dirigido, la estrategia
 * es ir construyendo gradualmente el MST, donde
 * iterativamente se coloca la arista disponible con menor
 * peso y ademas no conecte 2 nodos que pertenezcan al
 * mismo componente.
 * Tiempo: O(E log E) Status: testeado
 * en ICPC LATAM 2017 - Imperial Roads
 */

#include <../Data Structure/DSU.h>

int kruskal(int n, vector<tuple<int, int, int>>& e) {
  sort(ALL(e));
  DSU dsu;
  dsu.init(n);
  int mst = 0;
  for (auto& [w, u, v] : e)
    if (dsu.get(u) != dsu.get(v)) {
      mst += w;
      dsu.join(u, v);
      if (--n == 1) break;
    }
  return mst;
}
```

6.15 Prim

```
/**
 * Descripcion: tiene como principal funcion calcular la
 * suma del peso de las aristas del arbol minimo de
 * expansion (MST) de un grafo, la estrategia es ir
 * construyendo gradualmente el MST, se inicia con un nodo
 * arbitrario y se agregan sus aristas con nodos que no
 * hayan sido agregados con anterioridad y se va tomando
 * la de menor peso hasta completar el MST.
 * Tiempo: O(E
 * log E)
 */
int prim(vector<vector<pi>>& g) {
  vector<bool> taken(SZ(g), 0);
  priority_queue<pi> pq;
  auto process = [&](int u) -> void {
    taken[u] = 1;
    for (auto& [v, w] : g[u])
      if (!taken[v]) pq.push({-w, v});
  };
  process(0);
  int totalWeight = 0, takenEdges = 0;
  while (!pq.empty() && takenEdges != SZ(g) - 1) {
    auto [w, u] = pq.top();
    pq.pop();
    if (taken[u]) continue;
    totalWeight -= w;
    process(u);
    ++takenEdges;
  }
  return totalWeight;
}
```

6.16 Hierholzer

```
/*
 * Descripcion: busca un camino euleriano en el grafo
 * dado. Un camino euleriano se define como el recorrido
 * de un grafo que visita cada arista del grafo
 * exactamente una vez Un grafo no dirigido es euleriano
 * si, y solo si: es conexo y todos los vertices tienen un
 * grado par Un grafo dirigido es euleriano si, y solo si:
 * es conexo y todos los vertices tienen el mismo numero
 * de aristas entrantes y salientes. Si hay, exactamente,
 * un vertice u que tenga una arista saliente adicional y,
 * exactamente, un vertice v que tenga una arista entrante
 * adicional, el grafo contara con un camino euleriano de
 * u a v
 * Tiempo: O(E)
 */

int N;
vector<vi> graph;  // Grafo dirigido

vi hierholzer(int s) {
  vi ans, idx(N, 0), st;
  st.pb(s);
  while (!st.empty()) {
    int u = st.back();
    if (idx[u] < (int)graph[u].size()) {
      st.pb(graph[u][idx[u]]);
      ++idx[u];
    } else {
      ans.pb(u);
      st.pop_back();
    }
  }
  reverse(all(ans));
  return ans;
}
```

6.17 Topological Sort

```
/**
 * Descripcion: algoritmo para obtener el orden topologico
 * de un grafo dirigido, definido como el ordenamiento de
 * sus vertices tal que para cada arista (u, v), u este
 * antes que v en el ordenamiento. Si existen ciclos,
 * dicho ordenamiento no existe.
 * Tiempo: O(V + E)
 */

int V;
vi graph[MAXN];
vi sorted_nodes;
bool visited[MAXN];

void dfs(int u) {
  visited[u] = true;
  for (auto v : graph[u])
    if (!visited[v]) dfs(v);
  sorted_nodes.push(u);
}

void toposort() {
  for (int i = 0; i < V; i++)
    if (!visited[i]) dfs(i);
  reverse(ALL(sorted_nodes));

  assert(sorted_nodes.size() == V);
}

void lexicographic_toposort() {
  priority_queue<int> q;
  for (int i = 0; i < V; i++)
    if (in_degree[i] == 0) q.push(-i);

  while (!q.empty()) {
```

```
    int u = -q.top();
    q.pop();
    sorted_nodes.push_back(u);
    for (int v : graph[u]) {
      in_degree[v]--;
      if (in_degree[v] == 0) q.push(-v);
    }
  }

  assert(sorted_nodes.size() == V);
}
```

6.18 Dinic

```
/**
 * Descripcion: Algoritmo de Flujo Máximo.
 *
 * Uso:
 * Dinic dinic(N); dinic.addEdge(u, v, cap);
 * ll flujo = dinic.calc(s, t);
 *
 * Tiempo: O(VE \log U) usando "capacity scaling" con U =
 * max(cap). O(V^2 E) clásico sin iterar L.
 *         O(\min(E^{1/2}, V^{2/3})E) para redes
 * unitarias, y O(E\sqrt{V}) para matching bipartito.
 */

struct Dinic {
  struct Edge {
    int to, rev;
    ll c, oc;
    ll flow() {
      return max(oc - c, 0LL);
    }  // if you need flows
  };
  vi lvl, ptr, q;
  vector<vector<Edge>> adj;
  Dinic(int n) : lvl(n), ptr(n), q(n), adj(n) {}
  void addEdge(int a, int b, ll c, ll rcap = 0) {
    adj[a].push_back({b, SZ(adj[b]), c, c});
    adj[b].push_back({a, SZ(adj[a]) - 1, rcap, rcap});
  }
  ll dfs(int v, int t, ll f) {
    if (v == t || !f) return f;
    for (int& i = ptr[v]; i < SZ(adj[v]); i++) {
      Edge& e = adj[v][i];
      if (lvl[e.to] == lvl[v] + 1)
        if (ll p = dfs(e.to, t, min(f, e.c))) {
          e.c -= p, adj[e.to][e.rev].c += p;
          return p;
        }
    }
    return 0;
  }
  ll calc(int s, int t) {
    ll flow = 0;
    q[0] = s;
    FOR(L, 0, 31)
    do {  // 'int L=30' maybe faster for random data
      lvl = ptr = vi(SZ(q));
      int qi = 0, qe = lvl[s] = 1;
      while (qi < qe && !lvl[t]) {
        int v = q[qi++];
        for (Edge e : adj[v])
          if (!lvl[e.to] && e.c >> (30 - L))
            q[qe++] = e.to, lvl[e.to] = lvl[v] + 1;
      }
      while (ll p = dfs(s, t, LLONG_MAX)) flow += p;
    }
    while (lvl[t]);
    return flow;
  }
  bool leftOfMinCut(int a) { return lvl[a] != 0; }
};
```

6.19 Min Cost Max Flow

```
/**
 * Descripcion: Min-cost max-flow.
 *  Si los costos pueden ser negativos, llamar a setpi
 * antes que calc, pero nota que ciclos de costo negativo
 * no son permitidos. Para obtener el flujo, revisar solo
 * valores positivos.
 * Tiempo: O(F E log(V)), donde F es el
 * max flow. O(VE) por setpi Status: testeado en SWERC
 * 2024 J
 */

#include <bits/extc++.h>

const ll INF = numeric_limits<ll>::max() / 4;

struct MCMF {
  struct edge {
    int from, to, rev;
    ll cap, cost, flow;
  };
  int N;
  vector<vector<edge>> ed;
  vi seen;
  vector<ll> dist, pi;
  vector<edge*> par;

  MCMF(int N)
      : N(N), ed(N), seen(N), dist(N), pi(N), par(N) {}

  void addEdge(int from, int to, ll cap, ll cost) {
    if (from == to) return;
    ed[from].push_back(
        edge{from, to, int(ed[to].size()), cap, cost, 0});
    ed[to].push_back(edge{
        to, from, int(ed[from].size()) - 1, 0, -cost, 0});
  }

  void path(int s) {
    fill(seen.begin(), seen.end(), 0);
    fill(dist.begin(), dist.end(), INF);
    dist[s] = 0;
    ll di;

    __gnu_pbds::priority_queue<pair<ll, int>> q;
    vector<decltype(q)::point_iterator> its(N);
    q.push({0, s});

    while (!q.empty()) {
      s = q.top().second;
      q.pop();
      seen[s] = 1;
      di = dist[s] + pi[s];
      for (edge& e : ed[s])
        if (!seen[e.to]) {
          ll val = di - pi[e.to] + e.cost;
          if (e.cap - e.flow > 0 && val < dist[e.to]) {
            dist[e.to] = val;
            par[e.to] = &e;
            if (its[e.to] == q.end())
              its[e.to] = q.push({-dist[e.to], e.to});
            else
              q.modify(its[e.to], {-dist[e.to], e.to});
          }
        }
    }
    for (int i = 0; i < N; ++i)
      pi[i] = min(pi[i] + dist[i], INF);
  }

  pair<ll, ll> calc(int s, int t) {
    ll totflow = 0, totcost = 0;
    while (path(s), seen[t]) {
      ll fl = INF;
      for (edge* x = par[t]; x; x = par[x->from])
        fl = min(fl, x->cap - x->flow);

      totflow += fl;
      for (edge* x = par[t]; x; x = par[x->from]) {
        x->flow += fl;
        ed[x->to][x->rev].flow -= fl;
      }
    }
    for (int i = 0; i < N; ++i)
      for (edge& e : ed[i]) totcost += e.cost * e.flow;
    return {totflow, totcost / 2};
  }

  void setpi(int s) {
    fill(pi.begin(), pi.end(), INF);
    pi[s] = 0;
    int it = N, ch = 1;
    ll v;
    while (ch-- && it--)
      for (int i = 0; i < N; ++i)
        if (pi[i] != INF)
          for (edge& e : ed[i])
            if (e.cap)
              if ((v = pi[i] + e.cost) < pi[e.to])
                pi[e.to] = v, ch = 1;
    assert(it >= 0);  // ciclo de costo negativo
  }
};
```

6.20 Push Relabel

```
/**
 * Descripcion: algoritmo push-relabel para calcular el
 * flujo maximo en un grafo, bastante rapido en la
 * practica
 * Tiempo: $O(V^2\sqrt E)$
 */

template <typename T>
struct PushRelabel {
  struct Edge {
    int dest, back;
    T f, c;
  };
  vector<vector<Edge>> g;
  vector<T> ec;
  vector<Edge*> cur;
  vector<vi> hs;
  vi H;
  PushRelabel(int n)
      : g(n), ec(n), cur(n), hs(2 * n), H(n) {}
  void addEdge(int s, int t, T cap, T rcap = 0) {
    if (s == t) return;
    g[s].push_back({t, SZ(g[t]), 0, cap});
    g[t].push_back({s, SZ(g[s]) - 1, 0, rcap});
  }
  void addFlow(Edge& e, T f) {
    Edge& back = g[e.dest][e.back];
    if (!ec[e.dest] && f) hs[H[e.dest]].push_back(e.dest);
    e.f += f;
    e.c -= f;
    ec[e.dest] += f;
    back.f -= f;
    back.c += f;
    ec[back.dest] -= f;
  }
  T calc(int s, int t) {
    int v = SZ(g);
    H[s] = v;
    ec[t] = 1;
    vi co(2 * v);
    co[0] = v - 1;
    FOR(i, 0, v) cur[i] = g[i].data();
    for (Edge& e : g[s]) addFlow(e, e.c);
    for (int hi = 0;;) {
      while (hs[hi].empty())
```

```
        if (!hi--) return -ec[s];
      int u = hs[hi].back();
      hs[hi].pop_back();
      while (ec[u] > 0)
        if (cur[u] ==
            g[u].data() + SZ(g[u])) {  // discharge u
          H[u] = 1e9;
          for (Edge& e : g[u])
            if (e.c && H[u] > H[e.dest] + 1)
              H[u] = H[e.dest] + 1, cur[u] = &e;
          if (++co[H[u]], !--co[hi] && hi < v)
            FOR(i, 0, v)
            if (hi < H[i] && H[i] < v)-- co[H[i]],
                H[i] = v + 1;
          hi = H[u];
        } else if (cur[u]->c &&
                   H[u] == H[cur[u]->dest] + 1)
          addFlow(*cur[u], min(ec[u], cur[u]->c));
        else
          ++cur[u];
    }
  }
  bool leftOfMinCut(int a) { return H[a] >= SZ(g); }
};
```

6.21 Bellman-Ford

```
/**
 * Descripcion: calcula el costo minimo para ir de un nodo
 * hacia todos los demas alcanzables. Puede detectar
 * ciclos negativos, dando una ultima pasada y revisando
 * si alguna distancia se acorta.
 * Tiempo: O(VE)
 */

void bellmanFord(vector<array<int, 3>>& e, int n) {
  vi d(n, INF);
  d[x] = 0;
  for (int i = 0; i < n; i++)
    for (auto& [u, v, w] : e) d[v] = min(d[v], d[u] + w);
  for (auto& [u, v, w] : e)
    if (d[u] != INF && d[u] + w < d[v]) {
      // neg cycle: all nodes reachable from u have -INF
      // distance. To reconstruct neg cycle save "prev" of
      // each node, go up from u until repeating a node.
      // this node and all nodes between the two
      // occurences form a neg cycle
    }
}
```

6.22 Dijkstra

```
/**
 * Descripcion: calcula el costo minimo para
 * ir desde un nodo hacia todos los demas.
 *
 * Tiempo: O(E log V)
 */
vi dijkstra(vector<vector<pi>>& g, int x) {
  int n = SZ(g);
  vi d(n);
  FOR(i, 0, n) d[i] = INF;
  d[x] = 0;
  priority_queue<pi> pq;
  pq.emplace(0, x);
  while (!pq.empty()) {
    auto [du, u] = pq.top();
    pq.pop();
    du *= -1;
    if (du > d[u]) continue;
    for (auto& [v, w] : g[u])
      if (du + w < d[v]) {
        d[v] = du + w;
        pq.emplace(-d[v], v);
      }
  }
  return d;
}
// Para tener a lo mucho V elementos en la PQ
vi dijkstra(vector<vector<pi>>& g, int st) {
  int n = SZ(g);
  vi d(n);
  set<pi> pq;
  FOR(i, 0, n) pq.emplace(d[i], i), d[i] = INF;
  while (!pq.empty()) {
    auto [du, u] = *pq.begin();
    pq.erase(pq.begin());
    for (auto& [v, w] : g[u])
      if (du + w < d[v]) {
        pq.erase(pq.find({d[v], v}));
        d[v] = du + w;
        pq.emplace(d[v], v);
      }
  }
  return d;
}
```

6.23 Floyd-Warshall

```
/**
 * Descripcion: algoritmo de Floyd-Warshall para calcular
 * la minima distancia entre cada par de nodos, si no se
 * requiere recalcular el camino, ignorar p. Retorna un
 * par (g, p), en donde g es una matriz modificada en la
 * que g[i][j] es el costo minimo para llegar desde el
 * nodo i al nodo j y p es el nodo anterior en dicho
 * camino, utilizado para recalcular la ruta.
 * Tiempo:
 * O(V^3)
 */

pair<vector<vi>, vi> floydWarshall(vector<vi> g) {
  int n = SZ(g);
  vector<vi> p(n, vi(n));
  FOR(i, 0, n) FOR(j, 0, n) p[i][j] = i;
  FOR(k, 0, n)
  FOR(i, 0, n)
      FOR(j, 0, n) if (g[i][k] + g[k][j] < g[i][j]) {
    p[i][j] = p[k][j];
    g[i][j] = min(g[i][j], g[i][k] + g[k][j]);
  }
  return {g, p};
}
void printPath(vector<vi>& p, int i, int j) {
  if (i != j) printPath(p, i, p[i][j]);
  cout << j << " ";
}
```

6.24 Centroid Decomposition

```
/**
 * Descripcion: descompone un arbol recursivamente
 * eliminando su centroide, formando el "Arbol de
 * Centroides" (CD). Propiedades clave del CD:
 *   1. Su altura maxima es log(V).
 *   2. Para cualquier par de nodos (u, v) en el arbol
 * original, el LCA(u, v) en el CD es un nodo en el camino
 * de u a v. 'fat[u]' guarda el padre de 'u' en el CD (-1
 * si es la raiz). (Tip: Usar vector<basic_string<int>> en
 * lugar de vector<vector<int>> para la lista de
 * adyacencia puede mejorar los tiempos por SSO).
 * Tiempo:
 * O(V log V) Status: testeado en Codeforces 321C
 */

struct CD {
  int n;
  vector<vector<int>> adj;
  vector<int> sz, fat;
  vector<bool> tk;

  CD(const vector<vector<int>>& adj)
      : n(SZ(adj)),
        adj(adj),
        sz(n),
        fat(n, -1),
        tk(n, false) {
    decompose(0, getSz(0, -1));
  }

  int getSz(int u, int p) {
    sz[u] = 1;
    for (int v : adj[u])
      if (v != p && !tk[v]) sz[u] += getSz(v, u);
    return sz[u];
  }

  void decompose(int u, int globalSz, int p = -1) {
    for (int v : adj[u])
      if (!tk[v] && 2 * sz[v] >= globalSz) {
        sz[u] = 0;
        decompose(v, globalSz, p);
        return;
      }
    tk[u] = true;
    fat[u] = p;
    for (int v : adj[u])
      if (!tk[v]) decompose(v, getSz(v, -1), u);
  }
};
```

6.25 Euler Tour

```
/**
 * Descripcion: utilizando una DFS, es posible aplanar un
 * arbol, esto se logra guardando en que momento entra y
 * sale cada nodo, apoyandonos de una estructura para
 * consultas de rango es muy util para consultas sobre un
 * subarbol: saber la suma de todos los nodos en el, el
 * nodo con menor valor, etc.
 * Tiempo: O(n)
 */
vi g[maxn];
int in[maxn], out[maxn], t = 0;
void dfs(int u, int p) {
  in[u] = ++t;
  for (auto& v : g[u])
    if (v != p) dfs(v, u);
  out[u] = t;
}
```

6.26 HLD

```
/*
 * Heavy-Light Decomposition
 * Descripcion: descompone un arbol en caminos pesados y
 * aristas ligeras de tal manera que un camino de
 * cualquier hoja a la raiz contiene a lo mucho log(n)
 * aristas ligeras. Raiz debe ser 0 Si el valor lo
 * contiene las aristas, asignar VALS_EDGES true
 * Tiempo:
 * O((log N)^2)
 */

// Incluir Lazy Segment Tree

template <bool VALS_EDGES>
```

```
struct HLD {
  int N, tim = 0;
  vector<vi> adj;
  vi par, siz, head, pos;
  LazySegmentTree<int> tree;
  HLD(vector<vi> adj_)
      : N(sz(adj_)),
        adj(adj_),
        par(N, -1),
        siz(N, 1),
        head(N),
        pos(N) {
    tree.init(N);
    dfsSz(0);
    dfsHld(0);
  }
  void dfsSz(int v) {
    if (par[v] != -1)
      adj[v].erase(find(ALL(adj[v]), par[v]));
    for (int& u : adj[v]) {
      par[u] = v;
      dfsSz(u);
      siz[v] += siz[u];
      if (siz[u] > siz[adj[v][0]]) swap(u, adj[v][0]);
    }
  }
  void dfsHld(int v) {
    pos[v] = tim++;
    for (int u : adj[v]) {
      head[u] = (u == adj[v][0] ? head[v] : u);
      dfsHld(u);
    }
  }
  template <class B>
  void process(int u, int v, B op) {
    for (; head[u] != head[v]; v = par[head[v]]) {
      if (pos[head[u]] > pos[head[v]]) swap(u, v);
      op(pos[head[v]], pos[v] + 1);
    }
    if (pos[u] > pos[v]) swap(u, v);
    op(pos[u] + VALS_EDGES, pos[v] + 1);
  }
  void modifyPath(int u, int v, int val) {
    process(u, v,
            [&](int l, int r) { tree.add(l, r, val); });
  }
  int queryPath(int u, int v) {
    int res = -1e9;
    process(u, v, [&](int l, int r) {
      res = max(res, tree.query(l, r));
    });
    return res;
  }
  int querySubtree(int v) {  // modifySubtree es similar
    return tree.query(pos[v] + VALS_EDGES,
                      pos[v] + siz[v]);
  }
};
```

6.27 LCA

```
/**
 * Descripcion: ancestro comun mas bajo (LCA) usando
 * binary lifting. Responde consultas en O(log n).
 *
 * Tiempo: O(n log n) en construccion, O(log n) por
 * consulta. (Tip: Usar vector<basic_string<int>> en lugar
 * de vector<vector<int>> para la lista de adyacencia
 * puede mejorar el tiempo por SSO).
 * Uso: LCA lca(adj);
 *   int ans = lca.query(u, v);
 * Status: testeado en ICPC LATAM 2017 - Imperial Roads
 */
struct LCA {
  int n, maxlog;
  vector<vector<int>> jmp;
  vector<int> d;

  LCA(const vector<vector<int>>& adj, int root = 0) {
    n = SZ(adj);
    maxlog = __lg(n) + 2;
    jmp.assign(maxlog, vector<int>(n, -1));
    d.assign(n, 0);
    auto dfs = [&](auto& self, int u, int p) -> void {
      jmp[0][u] = p;
      for (int i = 1; i < maxlog; i++) {
        if (jmp[i - 1][u] != -1) {
          jmp[i][u] = jmp[i - 1][jmp[i - 1][u]];
        }
      }
      for (int v : adj[u]) {
        if (v != p) {
          d[v] = d[u] + 1;
          self(self, v, u);
        }
      }
    };
    dfs(dfs, root, -1);
  }

  int query(int u, int v) {
    if (d[u] < d[v]) swap(u, v);
    int dist = d[u] - d[v];
    for (int i = maxlog - 1; i >= 0; i--) {
      if ((dist >> i) & 1) u = jmp[i][u];
    }
    if (u == v) return u;
    for (int i = maxlog - 1; i >= 0; i--) {
      if (jmp[i][u] != jmp[i][v]) {
        u = jmp[i][u];
        v = jmp[i][v];
      }
    }
    return jmp[0][u];
  }

  int dist(int u, int v) {
    return d[u] + d[v] - 2 * d[query(u, v)];
  }
};
```

7 Strings

7.1 Aho Corasick

```
/*
 * Descripcion: Construye un automata que te permite
 * buscar rapidamente multiples patrones en un texto.
 * Uso:
 * - Se inicializa con AhoCorasick ac(patrones)
 * - find(word): regresa por cada posicion de word el
 * indice de la palabra mas larga que termina ahi o -1 si
 * no existe
 * - findAll(pat, word) encuentra todas las palabras que
 * empiezan por cada posicion de word (las mas cortas
 * primero) Notas:
 * - Patrones duplicados son permitidos (afecta en tiempo)
 * - Patrones vacios no permitidos
 * - El nodo de inicio del automata esta en el indice 0
 * - Para encontrar las palabras mas grande en cada
 * posicion, invierte la entrada
 * Tiempo:
 * - Contruccion: O(26N)
 * - find(x): O(N)
 * - findAll: O(NM) o hasta O(N sqrt(N)) si no hay
 * patrones duplicados
 */

struct AhoCorasick {
  enum { alpha = 26, first = 'a' };
  struct Node {
    int back, next[alpha], start = -1, end = -1,
                           nmatches = 0;
    Node(int v) { memset(next, v, sizeof(next)); }
  };
  vector<Node> N;
  vi backp;
  void insert(string& s, int j) {
    assert(!s.empty());
    int n = 0;
    for (char c : s) {
      int& m = N[n].next[c - first];
      if (m == -1) {
        n = m = SZ(N);
        N.emplace_back(-1);
      } else
        n = m;
    }
    if (N[n].end == -1) N[n].start = j;
    backp.push_back(N[n].end);
    N[n].end = j;
    N[n].nmatches++;
  }
  AhoCorasick(vector<string>& pat) : N(1, -1) {
    FOR(i, 0, SZ(pat))
    insert(pat[i], i);
    N[0].back = SZ(N);
    N.emplace_back(0);

    queue<int> q;
    for (q.push(0); !q.empty(); q.pop()) {
      int n = q.front(), prev = N[n].back;
      FOR(i, 0, alpha) {
        int &ed = N[n].next[i], y = N[prev].next[i];
        if (ed == -1)
          ed = y;
        else {
          N[ed].back = y;
          (N[ed].end == -1 ? N[ed].end
                           : backp[N[ed].start]) =
              N[y].end;
          N[ed].nmatches += N[y].nmatches;
          q.push(ed);
        }
      }
    }
  }
  vi find(string word) {
    int n = 0;
    vi res;  // ll count = 0;
    for (char c : word) {
      n = N[n].next[c - first];
      res.push_back(N[n].end);
      // count += N[n].nmatches;
    }
    return res;
  }
  vector<vi> findAll(vector<string>& pat, string word) {
    vi r = find(word);
    vector<vi> res(SZ(word));
    FOR(i, 0, SZ(word)) {
      int ind = r[i];
      while (ind != -1) {
        res[i - SZ(pat[ind]) + 1].push_back(ind);
        ind = backp[ind];
      }
    }
    return res;
  }
};
```

7.2 Dynamic Aho Corasick

```
/*
 * Descripcion: Si tenemos N cadenas en el diccionario,
 * mantenga log(N) Aho Corasick automatas. El i-esimo
 * automata contiene las primeras 2^k cadenas no incluidas
 * en el automatas anteriores. Por ejemplo, si tenemos N =
 * 19, necesitamos 3 automatas: {s[1]...s[16]},
 * {s[17]...s[18]} y {s[19]}. Para responder a la
 * consulta, podemos atravesar los automatas logN.
 * utilizando la cadena de consulta dada.
 * Para manejar la insercion, primero construya un
 * automata usando una sola cadena y luego Si bien hay dos
 * automatas con el mismo numero de cadenas, los
 * fusionamos mediante un nuevo automata usando fuerza
 * bruta. Para manejar la eliminacion, simplemente
 * insertamos un valor -1 para almacenar en los puntos
 * finales de cada cadena agregada.
 * Tiempo:
 * O(m*log(numero_de_inserciones))
 */

class AhoCorasick {
 public:
  struct Node {
    map<char, int> ch;
    vector<int> accept;
    int link = -1;
    int cnt = 0;

    Node() = default;
  };

  vector<Node> states;
  map<int, int> accept_state;

  explicit AhoCorasick() : states(1) {}

  void insert(const string& s, int id = -1) {
    int i = 0;
    for (char c : s) {
      if (!states[i].ch.count(c)) {
        states[i].ch[c] = states.size();
        states.emplace_back();
      }
      i = states[i].ch[c];
    }
    ++states[i].cnt;
    states[i].accept.push_back(id);
    accept_state[id] = i;
  }
  void clear() {
    states.clear();
    states.emplace_back();
  }

  int get_next(int i, char c) const {
    while (i != -1 && !states[i].ch.count(c))
      i = states[i].link;
    return i != -1 ? states[i].ch.at(c) : 0;
  }

  void build() {
    queue<int> que;
    que.push(0);
    while (!que.empty()) {
      int i = que.front();
      que.pop();

      for (auto [c, j] : states[i].ch) {
        states[j].link = get_next(states[i].link, c);
        states[j].cnt += states[states[j].link].cnt;

        auto& a = states[j].accept;
        auto& b = states[states[j].link].accept;
        vector<int> accept;
        set_union(a.begin(), a.end(), b.begin(), b.end(),
                  back_inserter(accept));
        a = accept;

        que.push(j);
      }
    }
  }

  long long count(const string& str) const {
    long long ret = 0;
    int i = 0;
    for (auto c : str) {
      i = get_next(i, c);
      ret += states[i].cnt;
    }
    return ret;
  }

  // list of (id, index)
  vector<pair<int, int>> match(const string& str) const {
    vector<pair<int, int>> ret;
    int i = 0;
    for (int k = 0; k < (int)str.size(); ++k) {
      char c = str[k];
      i = get_next(i, c);
      for (auto id : states[i].accept) {
        ret.emplace_back(id, k);
      }
    }
    return ret;
  }
};

class DynamicAhoCorasick {
  vector<vector<string>> dict;
  vector<AhoCorasick> ac;

 public:
  void insert(const string& s) {
    int k = 0;
    while (k < (int)dict.size() && !dict[k].empty()) ++k;
    if (k == (int)dict.size()) {
      dict.emplace_back();
      ac.emplace_back();
    }

    dict[k].push_back(s);
    ac[k].insert(s);

    for (int i = 0; i < k; ++i) {
      for (auto& t : dict[i]) {
        ac[k].insert(t);
      }
```

```
      dict[k].insert(dict[k].end(), dict[i].begin(),
                     dict[i].end());
      ac[i].clear();
      dict[i].clear();
    }

    ac[k].build();
  }

  long long count(const string& str) const {
    long long ret = 0;
    for (int i = 0; i < (int)ac.size(); ++i)
      ret += ac[i].count(str);
    return ret;
  }
};
```

7.3 Evil Hashing

```
/*
 * Evil Hashing
 * Descripcion: utiliza aritmetica modulo 2^64 - 1, el
 * doble de lento que mod 2^64 y con mas codigo, pero
 * funciona bien con casos malvados (como Thue-Morse,
 * donde ABBA... y BAAB... de longitud 2^10 tienen el
 * mismo hash mod 2^64) Utilizar "using H = ull;" si se
 * cree que los casos son aleatorios, o trabaja mod 10^9+7
 * si la paradoja del cumpleanos no es un problema.
 * Utiliza cosas de c++20.
 *
 * Uso:
 *  Normal.
 *     string s;
 *     HashInterval hi(s);
 *     hi.hashInterval(0, 5);
 *  Rabin-Karp.
 *     string s;
 *     getHashes(s, SZ(s));
 *     ....
 *
 * Tiempo: O(|s|)
 */

using ull = uint64_t;
struct H {
  ull x;
  H(ull x = 0) : x(x) {}
  H operator+(H o) { return x + o.x + (x + o.x < x); }
  H operator-(H o) { return *this + ~o.x; }
  H operator*(H o) {
    auto m = (__uint128_t)x * o.x;
    return H((ull)m) + (ull)(m >> 64);
  }
  ull get() const { return x + !~x; }
  bool operator==(H o) const { return get() == o.get(); }
  bool operator<(H o) const { return get() < o.get(); }
};
static const H C =
    (ll)1e11 + 3;  // (order ~ 3e9; random also ok)

struct HashInterval {
  vector<H> ha, pw;
  HashInterval(string& str) : ha(sz(str) + 1), pw(ha) {
    pw[0] = 1;
    FOR(i, 0, sz(str))
    ha[i + 1] = ha[i] * C + str[i], pw[i + 1] = pw[i] * C;
  }
  H hashInterval(int a, int b) {  // hash [a, b)
    return ha[b] - ha[a] * pw[b - a];
  }
};

vector<H> getHashes(string& str, int length) {
  if (sz(str) < length) {
    return {};
  }
  H h = 0, pw = 1;
  FOR(i, 0, length) {
    h = h * C + str[i];
    pw = pw * C;
  }
  vector<H> ret = {h};
  FOR(i, length, sz(str)) {
    ret.push_back(h = h * C + str[i] -
                      pw * str[i - length]);
  }
  return ret;
}

H hashString(string& s) {
  H h{};
  for (char c : s) {
    h = h * C + c;
  };
  return h;
}
```

7.4 Hashing

```
/*
 * Hashing
 * Descripcion: El objetivo es convertir una cadena en un
 * numero entero para poder comparar cadenas en O(1)
 *
 * Tiempo: O(|s|)
 */

const int MX = 3e5 + 2;  // Tamano maximo del string S

inline int add(int a, int b, const int &mod) {
  return a + b >= mod ? a + b - mod : a + b;
}
inline int sbt(int a, int b, const int &mod) {
  return a - b < 0 ? a - b + mod : a - b;
}
inline int mul(int a, int b, const int &mod) {
  return 1ll * a * b % mod;
}

const int X[] = {257, 359};
const int MOD[] = {(int)1e9 + 7, (int)1e9 + 9};
vector<int> xpow[2];

struct hashing {
  vector<int> h[2];

  hashing(string &s) {
    int n = s.size();
    for (int j = 0; j < 2; ++j) {
      h[j].resize(n + 1);
      for (int i = 1; i <= n; ++i) {
        h[j][i] = add(mul(h[j][i - 1], X[j], MOD[j]),
                      s[i - 1], MOD[j]);
      }
    }
  }

  ll value(int l, int r) {  // hash [i, j)
    int a =
        sbt(h[0][r], mul(h[0][l], xpow[0][r - l], MOD[0]),
            MOD[0]);
    int b =
        sbt(h[1][r], mul(h[1][l], xpow[1][r - l], MOD[1]),
            MOD[1]);
    return (ll(a) << 32) + b;
  }
};

// Llamar la funcion antes del hashing
void calc_xpow(int mxlen = MX) {
  for (int j = 0; j < 2; ++j) {
    xpow[j].resize(mxlen + 1, 1);
    for (int i = 1; i <= mxlen; ++i) {
      xpow[j][i] = mul(xpow[j][i - 1], X[j], MOD[j]);
    }
  }
}
```

7.5 Hashing Dynamic

```
/*
 * Hashing Dinamico
 * Descripcion: Convierte strings en hashes para
 * compararlos eficientemente
 * - Util para comparar strings o un substring de este
 * - Tambien puede cambiar un caracter del string
 * eficientemente
 * Uso:
 * - hash.get(inicio, fin); [inicio, fin)
 *   comparar dos string hash.get(l, f) == hash.get(l, f)
 * - set(posicion, caracter) indexado en 0
 *   Cambia el caracter de una posicion en el string
 * Aplicaciones:
 * - Checar si substrings de un string son palindromos
 *
 * Complejidad:
 * - Construccion O(n log(n))
 * - Query y update O(log(n))
 */

#include <bits/stdc++.h>
// Pura gente del coach moy
using namespace std;

typedef long long ll;
typedef pair<ll, ll> ii;

const ll MOD = 998244353;
const ii BASE = {1e9 + 7, 1e9 + 9};

ii operator+(const ii a, const ii b) {
  return {(a.first + b.first) % MOD,
          (a.second + b.second) % MOD};
}

ii operator+(const ii a, const ll b) {
  return {(a.first + b) % MOD, (a.second + b) % MOD};
}

ii operator-(const ii a, const ii b) {
  return {(MOD + a.first - b.first) % MOD,
          (MOD + a.second - b.second) % MOD};
}

ii operator*(const ii a, const ii b) {
  return {(a.first * b.first) % MOD,
          (a.second * b.second) % MOD};
}

ii operator*(const ii a, const ll b) {
  return {(a.first * b) % MOD, (a.second * b) % MOD};
}

inline ll modpow(ll x, ll p) {
  if (!p) return 1;
  return (modpow(x * x % MOD, p >> 1) * (p % 2 ? x : 1)) %
         MOD;
}

inline ll modinv(ll x) { return modpow(x, MOD - 2); }

struct Hash_Bit {
  int N;
  string S;
  vector<ii> fen, pp, ipp;
```

```
  Hash_Bit(string S_) {
    S = S_;
    N = S.size();
    fen.resize(N + 1);
    pp.resize(N);
    ipp.resize(N);
    pp[0] = ipp[0] = {1, 1};
    const ii ibase = {modinv(BASE.first),
                      modinv(BASE.second)};

    for (int i = 1; i < N; i++) {
      pp[i] = pp[i - 1] * BASE;
      ipp[i] = ipp[i - 1] * ibase;
    }

    for (int i = 0; i < N; i++) {
      update(i, S[i]);
    }
  }

  void update(int i, ll x) {
    ii p = pp[i] * x;
    for (++i; i <= N; i += i & -i) {
      fen[i] = fen[i] + p;
    }
  }

  ii query(int i) {
    ii ret = {0, 0};
    for (; i; i -= i & -i) {
      ret = (ret + fen[i]);
    }
    return ret;
  }

  void set(int idx, char c) {
    int d = (MOD + c - S[idx]) % MOD;
    S[idx] = c;
    update(idx, d);
  }

  ii get(int start, int end) {
    return (query(end) - query(start)) * ipp[start];
  }
};

int main() {
  string s;
  cin >> s;

  int sz = s.size();
  Hash_Bit hash(s);

  return 0;
}
```

7.6 KMP

```
/*
 * Descripcion: Calcula la funcion phi para un string S
 * donde phi[i] es la longitud del prefijo propio de S mas
 * largo de la subcadena S[0..i] el cual tambien es sufijo
 * de esta subcadena
 * Tiempo: O(|s| + |pat|)
 */

vi PI(const string& s) {
  vi p(SZ(s));
  FOR(i, 1, SZ(s)) {
    int g = p[i - 1];
    while (g && s[i] != s[g]) g = p[g - 1];
    p[i] = g + (s[i] == s[g]);
  }
  return p;
}

// Concatena s + \0 + pat para encontrar las ocurrencias
vi KMP(const string& s, const string& pat) {
  vi phi = PI(pat + '\0' + s), res;
  FOR(i, SZ(phi) - SZ(s), SZ(phi))
  if (phi[i] == SZ(pat)) res.push_back(i - 2 * SZ(pat));
  return res;
}

// A partir del phi de patron busca las ocurrencias en s
int KMP(const string& s, const string& pat) {
  vi phi = PI(pat);
  int matches = 0;
  for (int i = 0, j = 0; i < SZ(s); ++i) {
    while (j > 0 && s[i] != pat[j]) j = phi[j - 1];
    if (s[i] == pat[j]) ++j;
    if (j == SZ(pat)) {
      matches++;
      j = phi[j - 1];
    }
  }
  return matches;
}
```

7.7 KMPAutomaton

```
/*
 * Descripcion: Construye un automata del KMP donde el
 * estado es el valor actual de la funcion phi
 * Uso:
 * aut[state][nextCharacter]
 * Tiempo: O(|s|*C)
 */
vector<vector<int>> aut;
void compute_automaton(string s) {
  s += '#';
  int n = s.size();
  vector<int> phi = PI(s);
  aut.assign(n, vector<int>(26));
  FOR(i, 0, n) {
    FOR(c, 0, 26) {
      if (i > 0 && 'a' + c != s[i])
        aut[i][c] = aut[phi[i - 1]][c];
      else
        aut[i][c] = i + ('a' + c == s[i]);
    }
  }
}
```

7.8 Manacher

```
/*
 * Descripcion: para cada posicion de un string, calcula:
 * p[0][i] = la mitad de la longitud del palindromo par
 * mas grande alrededor de la posicion i
 * p[1][i] = la longitud del palindromo impar mas grande
 * alrededor de la posicion i (la mitad redondeado hacia
 * abajo)
 * Tiempo: O(N)
 */

array<vi, 2> manacher(const string& s) {
  int n = SZ(s);
  array<vi, 2> p = {vi(n + 1), vi(n)};
  FOR(z, 0, 2)
  for (int i = 0, l = 0, r = 0; i < n; i++) {
    int t = r - i + !z;
    if (i < r) p[z][i] = min(t, p[z][l + t]);
    int L = i - p[z][i], R = i + p[z][i] - !z;
    while (L >= 1 && R + 1 < n && s[L - 1] == s[R + 1])
      p[z][i]++, L--, R++;
    if (R > r) l = L, r = R;
  }
  return p;
}
```

7.9 Palindromic Tree

```
/*
 * Descripcion: Calcula un EerTree el cual es un arbol que
 * permite un rapido acceso a todos los palindromos
 * contenidos en un string como encontrar el palindromo
 * mas largo en un substring
 * Uso: Considerar a s como el
 * string original y p el palindromo asociado al nodo u
 * - et[u].l: la longitud de p
 * - et[u].cnt: es el numero de veces que aparece p como
 * el sufijo palindromo mas largo de un prefijo s[0:i].
 * Despues de llamar computeFrecuency() es el numero de
 * veces en el que p aparece en s
 * - et[u].link: es el nodo que correspondiente al
 * palindromo propio mas largo de p
 * Tiempo: O(N)
 */

constexpr short alpha = 26;
constexpr char offset = 'a';
struct state {
  int l, link, cnt;
  array<int, alpha> go;
  state() {
    l = cnt = 0;
    go.fill(0);
  }
};
struct PalindromicTree {
  int n = 2;
  int last = 1;
  vector<state> et;
  string s;
  PalindromicTree() {
    et.resize(2);
    et[0].link = et[1].link = 1;
    et[1].l = -1;
  }
  int palSuff(int x) {
    while (s[SZ(s) - 2 - et[x].l] != s.back())
      x = et[x].link;
    return x;
  }
  int add(char ch) {
    s.pb(ch);
    last = palSuff(last);
    bool new_pal = !et[last].go[ch - offset];
    if (new_pal) {
      et.pb(state());
      et[last].go[ch - offset] = n++;
      et.back().link =
          et[palSuff(et[last].link)].go[ch - offset];
      et.back().l = et[last].l + 2;
      if (et.back().l == 1) et.back().link = 0;
    }
    last = et[last].go[ch - offset];
    // Do something with last, maybe if new_pal
    et[last].cnt++;
    if (et[last].l == SZ(s)) last = et[last].link;
    return new_pal;
  }
  void computeFrequency() {
    for (int i = n - 1; i > 1; i--)
      et[et[i].link].cnt += et[i].cnt;
  }
};
```

7.10 Suffix Array

```
/*
 * Descripcion: construye un arreglo ordenado de todos los
 * sufijos de un string
 * Uso:
 * - SA[i]: es el indice de inicio del sufijo el cual es
 * el i-nesimo elemento del suffix array (el arreglo es de
 * tamano n+1 y SA[0] = n)
 * - LCP: es un arreglo que contiene el prefijo comun mas
 * largo entre los strings vecinos del suffix array LCP[i]
 * = LCP(sa[i], sa[i-1]), LCP[0] = 0
 * Tiempo: O(n log n)
 */

struct SuffixArray {
  vi SA, LCP;
  string S;
  int n;
  SuffixArray(string &s, int lim = 256)
      : S(s), n(SZ(s) + 1) {
    int k = 0, a, b;
    vi x(ALL(s) + 1), y(n), ws(max(n, lim)), rank(n);
    SA = LCP = y, iota(ALL(SA), 0);

    // Calcular SA
    for (int j = 0, p = 0; p < n;
         j = max(1, j * 2), lim = p) {
      p = j, iota(ALL(y), n - j);
      FOR(i, 0, n) {
        if (SA[i] >= j) y[p++] = SA[i] - j;
      }
      fill(ALL(ws), 0);
      FOR(i, 0, n) { ws[x[i]]++; }
      FOR(i, 1, lim) { ws[i] += ws[i - 1]; }
      for (int i = n; i--;) SA[--ws[x[y[i]]]] = y[i];
      swap(x, y), p = 1, x[SA[0]] = 0;
      FOR(i, 1, n) {
        a = SA[i - 1];
        b = SA[i],
        x[b] = (y[a] == y[b] && y[a + j] == y[b + j])
                   ? p - 1
                   : p++;
      }
    }

    // Calcular LCP (longest common prefix)
    FOR(i, 1, n) { rank[SA[i]] = i; }
    for (int i = 0, j; i < n - 1; LCP[rank[i++]] = k)
      for (k &&k--, j = SA[rank[i] - 1];
           s[i + k] == s[j + k]; k++);
  }

  /*
   * Retorna el lower_bound de la subcadena sub en el
   * Suffix Array
  * Tiempo: O(|sub| log n)
   */
  int lower(string &sub) {
    int l = 0, r = n - 1;
    while (l < r) {
      int mid = (l + r) / 2;
      int res = S.compare(SA[mid], SZ(sub), sub);
      (res >= 0) ? r = mid : l = mid + 1;
    }
    return l;
  }

  /*
   * Retorna el upper_bound de la subcadena sub en el
   * Suffix Array
  * Tiempo: O(|sub| log n)
   */
  int upper(string &sub) {
    int l = 0, r = n - 1;
    while (l < r) {
      int mid = (l + r) / 2;
      int res = S.compare(SA[mid], SZ(sub), sub);
      (res > 0) ? r = mid : l = mid + 1;
    }
    if (S.compare(SA[r], SZ(sub), sub) != 0) --r;
    return r;
  }

  /*
   * Busca si se encuentra la subcadena sub en el Suffix
   * Array
  * Tiempo: O(|sub| log n)
   */
  bool subStringSearch(string &sub) {
    int L = lower(sub);
    if (S.compare(SA[L], SZ(sub), sub) != 0) return 0;
    return 1;
  }

  /*
   * Cuenta la cantidad de ocurrencias de la subcadena sub
   * en el Suffix Array
  * Tiempo: O(|sub| log n)
   */
  int countSubString(string &sub) {
    return upper(sub) - lower(sub) + 1;
  }

  /*
   * Cuenta la cantidad de subcadenas distintas en el
   * Suffix Array
  * Tiempo: O(n)
   */
  ll countDistinctSubstring() {
    ll result = 0;
    FOR(i, 1, n) { result += ll(n - SA[i] - 1 - LCP[i]); }
    return result;
  }

  /*
   * Busca la subcadena mas grande que se encuentra en el
   * string T y S
  * Uso: Crear el SuffixArray con una cadena
   * de la concatenacion de T y S separado por un caracter
   * especial (T + '#' + S)
  * Tiempo: O(n)
   */
  string longestCommonSubstring(int lenS, int lenT) {
    int maximo = -1, indice = -1;
    FOR(i, 2, n) {
      if ((SA[i] > lenS && SA[i - 1] < lenS) ||
          (SA[i] < lenS && SA[i - 1] > lenS)) {
        if (LCP[i] > maximo) {
          maximo = LCP[i];
          indice = SA[i];
        }
      }
    }
    return S.substr(indice, maximo);
  }

  /*
   * A partir del Suffix Array se crea un Suffix Array
   * inverso donde la posicion i del string S devuelve la
   * posicion del sufijo S[i..n) en el Suffix Array
   *
  * Tiempo: O(n)
   */
  vi constructRSA() {
    vi RSA(n);
    FOR(i, 0, n) { RSA[SA[i]] = i; }
    return RSA;
  }
};
```

7.11 Suffix Automaton

```
/*
 * Descripcion: Construye un automata finito que reconoce
 * todos los sufijos de una cadena. len corresponde a la
 * longitud maxima de un sufijo con el estado actual, link
 * corresponde al estado del siguiente sufijo mas largo
 * del estado actual
 * Tiempo: O(n log sum)
 */

struct SuffixAutomaton {
  struct state {
    int len, link;
    map<char, int> next;
  };
  vector<state> st;
  int sz, last;
  SuffixAutomaton(int MAX = 1e5) {
    st.resize(MAX);
    last = st[0].len = 0;
    sz = 1;
    st[0].link = -1;
  }
  void extend(char c) {
    int k = sz++, p;
    st[k].len = st[last].len + 1;
    for (p = last; p != -1 && !st[p].next.count(c);
         p = st[p].link)
      st[p].next[c] = k;
    if (p == -1)
      st[k].link = 0;
    else {
      int q = st[p].next[c];
      if (st[p].len + 1 == st[q].len)
        st[k].link = q;
      else {
        int w = sz++;
        st[w].len = st[p].len + 1;
        st[w].next = st[q].next;
        st[w].link = st[q].link;
        for (; p != -1 && st[p].next[c] == q;
             p = st[p].link)
          st[p].next[c] = w;
        st[q].link = st[k].link = w;
      }
    }
    last = k;
  }
};
```

7.12 Suffix Tree

```
/**
 * Descripcion: Algoritmo de Ukkonen la construccion de
 * online arbol de sufijo. Cada nodo contiene indices
 * [l,r) en la cadena y una lista de nodos hijos. Los
 * sufijos estan dados por recorridos de este arbol
 * uniendo subcadenas [l,r). La raiz es 0 (tiene l = -1, r
 * = 0) los hijos inexistentes son -1 Para obtener un
 * arbol completo, agregue un simbolo ficticio al final
 *
 * Tiempo: O(26N)
 */

struct SuffixTree {
  enum { N = 200010, ALPHA = 26 };  // N ~ 2*maxlen+10
  int toi(char c) { return c - 'a'; }
  string a;  // v = cur node, q = cur position
  int t[N][ALPHA], l[N], r[N], p[N], s[N], v = 0, q = 0,
                                           m = 2;

  void ukkadd(int i, int c) {
  suff:
    if (r[v] <= q) {
      if (t[v][c] == -1) {
        t[v][c] = m;
        l[m] = i;
        p[m++] = v;
```

```
      v = s[v];
      q = r[v];
      goto suff;
    }
    v = t[v][c];
    q = l[v];
  }
  if (q == -1 || c == toi(a[q]))
    q++;
  else {
    l[m + 1] = i;
    p[m + 1] = m;
    l[m] = l[v];
    r[m] = q;
    p[m] = p[v];
    t[m][c] = m + 1;
    t[m][toi(a[q])] = v;
    l[v] = q;
    p[v] = m;
    t[p[m]][toi(a[l[m]])] = m;
    v = s[p[m]];
    q = l[m];
    while (q < r[m]) {
      v = t[v][toi(a[q])];
      q += r[v] - l[v];
    }
    if (q == r[m])
      s[m] = v;
    else
      s[m] = m + 2;
    q = r[v] - (q - r[m]);
    m += 2;
    goto suff;
  }
}

SuffixTree(string a) : a(a) {
  fill(r, r + N, SZ(a));
  memset(s, 0, sizeof s);
  memset(t, -1, sizeof t);
  fill(t[1], t[1] + ALPHA, 0);
  s[0] = 1;
  l[0] = l[1] = -1;
  r[0] = r[1] = p[0] = p[1] = 0;
  FOR(i, 0, SZ(a)) ukkadd(i, toi(a[i]));
}

// example: find longest common substring (uses ALPHA =
// 28)
pii best;
int lcs(int node, int i1, int i2, int olen) {
  if (l[node] <= i1 && i1 < r[node]) return 1;
  if (l[node] <= i2 && i2 < r[node]) return 2;
  int mask = 0,
      len = node ? olen + (r[node] - l[node]) : 0;
  FOR(c, 0, ALPHA)
  if (t[node][c] != -1) mask |=
      lcs(t[node][c], i1, i2, len);
  if (mask == 3) best = max(best, {len, r[node] - len});
  return mask;
}
static pii LCS(string s, string t) {
  SuffixTree st(s + (char)('z' + 1) + t +
                (char)('z' + 2));
  st.lcs(0, SZ(s), SZ(s) + 1 + SZ(t), 0);
  return st.best;
}
};
```

7.13 Trie

```
/*
 * Descripcion: Un trie es una estructura de datos de
 * arbol multidireccional que se utiliza para almacenar
 * cadenas en un alfabeto. La coincidencia de patrones se
 * puede realizar de manera eficiente usando trie
 * Tiempo:
 * O(n)
 */

struct TrieNode {
  unordered_map<char, TrieNode *> children;
  bool isEndOfWord;
  int numPrefix;

  TrieNode() : isEndOfWord(false), numPrefix(0) {}
};

class Trie {
 private:
  TrieNode *root;

 public:
  Trie() { root = new TrieNode(); }

  void insert(string word) {
    TrieNode *curr = root;
    for (char c : word) {
      if (curr->children.find(c) ==
          curr->children.end()) {
        curr->children[c] = new TrieNode();
      }
      curr = curr->children[c];
      curr->numPrefix++;
    }
    curr->isEndOfWord = true;
  }

  bool search(string word) {
    TrieNode *curr = root;
    for (char c : word) {
      if (curr->children.find(c) ==
          curr->children.end()) {
        return false;
      }
      curr = curr->children[c];
    }
    return curr->isEndOfWord;
  }

  bool startsWith(string prefix) {
    TrieNode *curr = root;
    for (char c : prefix) {
      if (curr->children.find(c) ==
          curr->children.end()) {
        return false;
      }
      curr = curr->children[c];
    }
    return true;
  }

  int countPrefix(string prefix) {
    TrieNode *curr = root;
    for (char c : prefix) {
      if (curr->children.find(c) ==
          curr->children.end()) {
        return 0;
      }
      curr = curr->children[c];
    }
    return curr->numPrefix;
  }
};
```

7.14 ZAlgorithm

```
/*
 * Descripcion: La Z-function es un arreglo donde el
 * elemento i es igual al numero mas grande de caracteres
 * que empiezan desde la posicion i que coincide con el
 * prefijo de S, excepto Z[0] = 0. (abacaba -> 0010301)
 *
 * Tiempo: O(|S|)
 */
vi Z(const string& S) {
  vi z(SZ(S));
  int l = -1, r = -1;
  FOR(i, 1, SZ(S)) {
    z[i] = i >= r ? 0 : min(r - i, z[i - l]);
    while (i + z[i] < SZ(S) && S[i + z[i]] == S[z[i]])
      z[i]++;
    if (i + z[i] > r) l = i, r = i + z[i];
  }
  return z;
}
```

8 Geometry

8.1 Coords

```
/*
 * Descripcion: Calcula con base en un plano un sistemas
 * de coordenadas para puntos 3D a 2D
 */

struct Coords {
  P3 o, dx, dy, dz;

  // De 3 puntos PQR en el plano
  Coords(P3 p, P3 q, P3 r) : o(p) {
    dx = (q - p).unit();
    dz = dx.cross(r - p).unit();
    dy = dz.cross(dx);
  }

  // De 4 puntos PQRS
  Coords(P3 p, P3 q, P3 r, P3 s)
      : o(p), dx(q - p), dy(r - p), dz(s - p) {}

  Point pos2D(P3 p) {
    return {(p - o).dot(dx), (p - o).dot(dy)};
  }
  P3 pos3D(P3 p) {
    return {(p - o).dot(dx), (p - o).dot(dy),
            (p - o).dot(dz)};
  }
};
```

8.2 Hull3D

```
/**
 * Descripcion: encuentra la envolvente convexa de un
 * conjunto de puntos 3D. Es el menor poliedro convexo que
 * contiene todos los puntos dados. Te retorna un vector
 * de caras en el que guarda el vector normal del plano y
 * los indices de los 3 puntos que forma el triangulo.
 *
 * Nota: Ningun conjunto de 4 puntos deben ser coplanar
 *
 * Tiempo: O(n^2)
 */

struct PR {
  void ins(int x) { (a == -1 ? a : b) = x; }
  void rem(int x) { (a == x ? a : b) = -1; }
  int cnt() { return (a != -1) + (b != -1); }
  int a, b;
};

struct F {
  P3 q;
  int a, b, c;
};

vector<F> hull3d(const vector<P3>& A) {
  assert(SZ(A) >= 4);
  vector<vector<PR>> E(SZ(A),
                       vector<PR>(SZ(A), {-1, -1}));
#define E(x, y) E[f.x][f.y]
  vector<F> FS;
  auto mf = [&](int i, int j, int k, int l) {
    P3 q = (A[j] - A[i]).cross((A[k] - A[i]));
    if (q.dot(A[l]) > q.dot(A[i])) q = q * -1;
    F f{q, i, j, k};
    E(a, b).ins(k);
    E(a, c).ins(j);
    E(b, c).ins(i);
    FS.push_back(f);
  };
  FOR(i, 0, 4)
    FOR(j, i + 1, 4) FOR(k, j + 1, 4)
      mf(i, j, k, 6 - i - j - k);

  FOR(i, 4, SZ(A)) {
    FOR(j, 0, SZ(FS)) {
      F f = FS[j];
      if (f.q.dot(A[i]) > f.q.dot(A[f.a])) {
        E(a, b).rem(f.c);
        E(a, c).rem(f.b);
        E(b, c).rem(f.a);
        swap(FS[j--], FS.back());
        FS.pop_back();
      }
    }
    int nw = SZ(FS);
    FOR(j, 0, nw) {
      F f = FS[j];
#define C(a, b, c) \
  if (E(a, b).cnt() != 2) mf(f.a, f.b, i, f.c);
      C(a, b, c);
      C(a, c, b);
      C(b, c, a);
    }
  }
  for (F& it : FS)
    if ((A[it.b] - A[it.a])
            .cross(A[it.c] - A[it.a])
            .dot(it.q) <= 0)
      swap(it.c, it.b);
  return FS;
};
```

8.3 Line3D

```
struct Line3D {
  P3 d, o;

  // De dos puntos P, Q
  Line3D(P3 p, P3 q) : d(q - p), o(p) {}

  // De dos planos p1, p2 (requiere double)
  Line3D(Plane p1, Plane p2) {
    d = p1.n.cross(p2.n);
    o = (p2.n * p1.d - p1.n * p2.d).cross(d) / d.sq();
  }

  // Distancia de un punto a la linea
  double sqDist(P3 p) {
    return d.cross(p - o).sq() / d.sq();
  }
  double dist(P3 p) { return sqrt(sqDist(p)); }

  // Compara dos puntos acorde a su posicion en la linea
  bool cmpProj(P3 p, P3 q) { return d.dot(p) < d.dot(q); }

  P3 proj(P3 p) {
    return o + d * (d.dot(p - o)) / d.sq();
  }
  P3 refl(P3 p) { return proj(p) * 2 - p; }

  // Retorna el punto de un interseccion de la linea con
  // el plano
  P3 inter(Plane p) {
    return o - d * (p.side(o)) / (p.n.dot(d));
  }
};

// Menor distancia entre dos lineas
double dist(Line3D l1, Line3D l2) {
  P3 n = l1.d.cross(l2.d);
  if (n == P3{0, 0, 0})  // Es paralelo
    return l1.dist(l2.o);
  return abs((l2.o - l1.o).dot(n)) / n.norm();
}

// Retorna el punto mas cercano de la linea L1 con la
// linea L2
P3 closestOnL1(Line3D l1, Line3D l2) {
  P3 n2 = l2.d.cross(l1.d.cross(l2.d));
  return l1.o +
         l1.d * ((l2.o - l1.o).dot(n2)) / (l1.d.dot(n2));
}

double angle(Line3D l1, Line3D l2) {
  return angle(l1.d, l2.d);
}
bool isParallel(Line3D l1, Line3D l2) {
  return l1.d.cross(l2.d) == P3{0, 0, 0};
}
bool isPerpendicular(Line3D l1, Line3D l2) {
  return l1.d.dot(l2.d) == 0;
}

// Entre un plano y una linea
double angle(Plane p, Line3D l) {
  return M_PI / 2 - angle(p.n, l.d);
}
bool isParallel(Plane p, Line3D l) {
  return p.n.dot(l.d) == 0;
}
bool isPerpendicular(Plane p, Line3D l) {
  return p.n.cross(l.d) == P3{0, 0, 0};
}
```

8.4 Plane

```
struct Plane {
  P3 n;
  T d;

  // De vector normal n y offset
  Plane(P3 n, T d) : n(n), d(d) {}

  // De vector normal n y punto P
  Plane(P3 n, P3 p) : n(n), d(n.dot(p)) {}

  // De 3 puntos no colineares PQR
  Plane(P3 p, P3 q, P3 r)
      : Plane((q - p).cross(r - p), p) {}

  // 0 si esta en el plano | > 0 sobre | < 0 debajo
  T side(P3 p) { return n.dot(p) - d; }

  double dist(P3 p) { return abs(side(p)) / n.norm(); }
  Plane translate(P3 t) { return {n, d + (n.dot(t))}; }

  // Mueve el plano perpendicularmente en direccion del
  // vector n
  Plane shiftUp(double dist) {
    return {n, d + dist * n.norm()};
  }

  P3 proj(P3 p) { return p - n * side(p) / n.sq(); }
  P3 refl(P3 p) { return p - n * 2 * side(p) / n.sq(); }
};

bool isParallel(Plane p1, Plane p2) {
  return p1.n.cross(p2.n) == P3{0, 0, 0};
}
bool isPerpendicular(Plane p1, Plane p2) {
  return (p1.n.dot(p2.n) == 0);
}
double angle(Plane p1, Plane p2) {
  return angle(p1.n, p2.n);
}

// Retorna la linea perpendicular a el plano que pasa por
// el punto O
Line3D perpThrough(Plane p, P3 o) {
  return Line3D(o, o + p.n);
}
```

```
// Retorna el plano perpendicular a la linea l que pasa
// por el punto O
Plane perpThrough(Line3D l, P3 o) {
  return Plane(l.d, o);
}
```

8.5 Point3D

```
typedef double T;

struct P3 {
  T x, y, z;

  // Operaciones Punto - Punto
  P3 operator+(P3 p) const {
    return {x + p.x, y + p.y, z + p.z};
  }
  P3 operator-(P3 p) const {
    return {x - p.x, y - p.y, z - p.z};
  }

  // Operacion Punto - Numero
  P3 operator*(T d) const {
    return {x * d, y * d, z * d};
  }
  P3 operator/(T d) const {
    return {x / d, y / d, z / d};
  }

  // Operaciones de comparacion para flotantes
  bool operator<(P3 p) const {
    return x < p.x - EPS ||
           (abs(x - p.x) <= EPS && y < p.y - EPS) ||
           (abs(x - p.x) <= EPS && abs(y - p.y) <= EPS &&
            z < p.z - EPS);
  }
  bool operator==(P3 p) const {
    return abs(x - p.x) <= EPS && abs(y - p.y) <= EPS &&
           abs(z - p.z) <= EPS;
  }
  bool operator!=(P3 p) const { return !(*this == p); }

  // Operaciones de comparacion para enteros
  bool operator<(P3 p) const {
    return tie(x, y, z) < tie(p.x, p.y, p.z);
  }
  bool operator==(P3 p) const {
    return tie(x, y, z) == tie(p.x, p.y, p.z);
  }

  // Operaciones generales
  T sq() { return x * x + y * y + z * z; }
  double norm() { return sqrt(sq()); }
  P3 unit() { return *this / norm(); }
  T dot(P3 p) { return x * p.x + y * p.y + z * p.z; }
  P3 cross(P3 p) {
    return {y * p.z - z * p.y, z * p.x - x * p.z,
            x * p.y - y * p.x};
  }
  P3 cross(P3 a, P3 b) {
    return (a - *this).cross(b - *this);
  }

  friend ostream &operator<<(ostream &os, P3 p) {
    return os << "(" << p.x << "," << p.y << "," << p.z
              << ")";
  }
};

double dist(P3 p1, P3 p2) {
  return sqrt((p1.x - p2.x) * (p1.x - p2.x) +
              (p1.y - p2.y) * (p1.y - p2.y) +
              (p1.z - p2.z) * (p1.z - p2.z));
}

bool areCollinear(P3 a, P3 b, P3 c) {
  P3 normal = (b - a).cross(c - a);
  return (normal.x == 0 && normal.y == 0 &&
          normal.z == 0);
}

// Si > 0 S esta sobre el plano PQR
// Si = 0 S esta en el plano PQR
// Si < 0 S esta bajo el plano PQR
// Nota: |orient| es igual a 6 veces el volumen de un
// tetraedro PQRS
T orient(P3 p, P3 q, P3 r, P3 s) {
  return ((q - p).cross(r - p)).dot(s - p);
}

// > 0 -> left | 0 -> aligned | < 0 -> right
T orientByNormal(P3 p, P3 q, P3 r, P3 n) {
  return ((q - p).cross(r - p)).dot(n);
}

// Angulo entre dos vectores [0, PI]
double angle(P3 v, P3 w) {
  double cosTheta = v.dot(w) / v.norm() / w.norm();
  return acos(max(-1.0, min(1.0, cosTheta)));
}
// Angulo aob
double angle(P3 o, P3 a, P3 b) {
  return angle(a - o, b - o);
}
```

8.6 Polyhedron

```
/*
 * Un poliedro es un cuerpo geométrico tridimensional
 * con caras planas que encierra un volumen finito
 */

// Retorna el vector area * 2 (evita decimales)
P3 vectorArea2(vector<P3> &p) {
  P3 S = {0, 0, 0};
  int n = SZ(p);
  FOR(i, 0, n) S = S + p[i].cross(p[(i + 1) % n]);
  return S;
}

double area(vector<P3> &p) {
  return vectorArea2(p).norm() / 2.0;
}

// Dado un vector de caras planas retorna el volumen
double volume(vector<vector<P3>> faces) {
  double vol6 = 0.0;
  for (auto face : faces) {
    vol6 += vectorArea2(face).dot(face[0]);
  }
  return abs(vol6) / 6.0;
}
```

8.7 Sphere

```
// Retorna el punto de la esfera centrada en el origen con
// la posicion representada por coordenadas de latitud y
// longitud
P3 sph(double r, double lat, double lon) {
  lat *= M_PI / 180, lon *= M_PI / 180;
  return {r * cos(lat) * cos(lon),
          r * cos(lat) * sin(lon), r * sin(lat)};
}

// Retorna el numero de puntos de interseccion de la linea
// en la esfera y los guarda en out
int sphereLine(P3 o, double r, Line3D l,
               pair<P3, P3> &out) {
  double h2 = r * r - l.sqDist(o);
  if (h2 < 0) return 0;  // La linea no toca la esfera
  P3 p = l.proj(o);
  P3 h = l.d * sqrt(h2) / l.d.norm();
  out = {p - h, p + h};
  return 1 + (h2 > 0);
}

// Retorna la distancia mas corta entre los puntos A y B
// de la esfera
double greatCircleDist(P3 o, double r, P3 a, P3 b) {
  return r * angle(a - o, b - o);
}

// Checa si solo existe un segmento mas corto entre A y B
// en la esfera
bool validSegment(P3 a, P3 b) {
  return a.cross(b) != P3{0, 0, 0} || a.dot(b) > 0;
}

// Retorna el punto de interseccion de los segmentos AB y
// CD que pasan a traves de una esfera
bool properInter(P3 a, P3 b, P3 c, P3 d, P3 &out) {
  P3 ab = a.cross(b), cd = c.cross(d);
  int oa = sgn(cd.dot(a));
  int ob = sgn(cd.dot(b));
  int oc = sgn(ab.dot(c));
  int od = sgn(ab.dot(d));
  out = ab.cross(cd) * od;
  return (oa != ob && oc != od && oa != oc);
}

// Checa si el punto P pasa por el segmento AB que pasan
// por la esfera
bool onSphSegment(P3 a, P3 b, P3 p) {
  P3 n = a.cross(b);
  if (n == P3{0, 0, 0})
    return a.cross(p) == P3{0, 0, 0} && a.dot(p) > 0;
  return n.dot(p) == 0 && n.dot(a.cross(p)) >= 0 &&
         n.dot(b.cross(p)) <= 0;
}

// Retorna el angulo que se forma en el vector AB y el
// vector AC
double angleSph(P3 a, P3 b, P3 c) {
  return angle(a.cross(b), a.cross(c));
}
double orientedAngleSph(P3 a, P3 b, P3 c) {
  if (a.cross(b).dot(c) >= 0) return angleSph(a, b, c);
  return 2 * M_PI - angleSph(a, b, c);
}

// Retorna el area que se forma en los puntos P con la
// esfera de radio r
double areaOnSphere(double r, vector<P3> p) {
  int n = SZ(p);
  double sum = -(n - 2) * M_PI;
  FOR(i, 0, n)
  sum += orientedAngleSph(p[(i + 1) % n], p[(i + 2) % n],
                          p[i]);
  return r * r * sum;
}
```

8.8 Circle

```
// Retorna el punto central del circulo que pasa por A,B,C
// Si se busca el radio solo sacar la distancia entre el
// centro y cualquier punto A,B,C
Point circumCenter(const Point& A, const Point& B,
                   const Point& C) {
  Point b = C - A, c = B - A;
  assert(!equal(b.cross(c), 0));
  return A + (b * c.sq() - c * b.sq()).perp() /
             b.cross(c) / 2;
}
```

```
// Retorna el punto que se encuentra en el circulo dado el
// angulo
Point circlePoint(Point c, double r, double ang) {
  return Point{c.x + cos(ang) * r, c.y + sin(ang) * r};
}

// Retorna el numero de intersecciones de la linea l con
// el circulo (o,r) y los pone en out. Si solo hay una
// interseccion el par de out es igual
int circleLine(Point o, double r, Line l,
               pair<Point, Point>& out) {
  double h2 = r * r - l.sqDist(o);
  if (greaterEqual(h2, 0)) {
    Point p = l.proj(o);
    Point h = l.v * sqrt(h2) / l.v.norm();
    out = {p - h, p + h};
  }
  return 1 + sgn(h2);
}

// Retorna las intersecciones entre dos circulos. Funciona
// igual que la interseccion con una linea
int circleCircle(Point o1, double r1, Point o2, double r2,
                 pair<Point, Point>& out) {
  Point d = o2 - o1;
  double d2 = d.sq();
  if (equal(d2, 0)) {
    assert(!equal(r1, r2));  // los circulos son iguales
    return 0;
  }
  double pd = (d2 + r1 * r1 - r2 * r2) / 2;
  double h2 = r1 * r1 - pd * pd / d2;
  if (greaterEqual(h2, 0)) {
    Point p = o1 + d * pd / d2,
          h = d.perp() * sqrt(h2 / d2);
    out = {p - h, p + h};
  }
  return 1 + sgn(h2);
}

// Retorna un booleano indicando si los dos circulos
// intersectan o no
bool circleCircle(Point o1, double r1, Point o2,
                  double r2) {
  double dx = o1.x - o2.x, dy = o1.y - o2.y, rs = r1 + r2;
  return dx * dx + dy * dy <= rs * rs;
}

// Retorna el area de la interseccion de un circulo con
// un poligono ccw
// Tiempo O(n)
#define arg(p, q) atan2(p.cross(q), p.dot(q))
double circlePoly(Point c, double r, vector<Point> ps) {
  auto tri =
      [&](Point p,
          Point q) {  // area de interseccion con cpq
        auto r2 = r * r / 2;
        Point d = q - p;
        auto a = d.dot(p) / d.sq(),
             b = (p.sq() - r * r) / d.sq();
        auto det = a * a - b;
        if (lessEqual(det, 0)) return arg(p, q) * r2;
        auto s = max(0., -a - sqrt(det)),
             t = min(1., -a + sqrt(det));
        if (less(t, 0) || greaterEqual(s, 1))
          return arg(p, q) * r2;
        Point u = p + d * s, v = p + d * t;
        return arg(p, u) * r2 + u.cross(v) / 2 +
               arg(v, q) * r2;
      };
  auto sum = 0.0;
  FOR(i, 0, SZ(ps))
  sum += tri(ps[i] - c, ps[(i + 1) % SZ(ps)] - c);
  return sum;
}

// Retorna el numero de tangentes de tipo especifico
// (inner, outer)
// * Si hay 2 tangentes. Out se llena con 2 pares de
// puntos:
//   los pares de puntos de tangencia de cada circulo
//   (P1,P2)
// * Si solo hay 1 tangente, los circulo son tangentes en
// algun
//   punto P, out contiene P 4 veces y la linea tangente
//   puede ser encontrada como line(o1,p).perp(p)
// * Si hay 0 tangentes, no hace nada
// * Si los circulos son identicos, aborta
int tangents(Point o1, double r1, Point o2, double r2,
             bool inner,
             vector<pair<Point, Point>>& out) {
  if (inner) r2 = -r2;
  Point d = o2 - o1;
  double dr = r1 - r2, d2 = d.sq(), h2 = d2 - dr * dr;
  if (equal(d2, 0) || less(h2, 0)) {
    assert(!equal(d2, 0));
    return 0;
  }
  for (int sign : {-1, 1}) {
    Point v = (d * dr + d.perp() * sqrt(h2) * sign) / d2;
    out.push_back({o1 + v * r1, o2 + v * r2});
  }
  return 1 + sgn(h2);
}
```

8.9 Closest Pair

```
/*
 * Descripcion: Dado un arreglo de N puntos en el plano,
 * encontrar el par de puntos con la menor distancia entre
 * ellos Utilizar con long long de preferencia
 * Tiempo: O(n
 * log n)
 */

typedef Point P;
pair<Point, Point> closest(vector<Point> &v) {
  set<Point> S;
  sort(ALL(v),
       [](Point a, Point b) { return a.y < b.y; });
  pair<ll, pair<Point, Point>> ret{LLONG_MAX,
                                   {P{0, 0}, P{0, 0}}};
  int j = 0;
  for (Point p : v) {
    Point d{1 + (ll)sqrt(ret.first), 0};
    while (v[j].y <= p.y - d.x) S.erase(v[j++]);
    auto lo = S.lower_bound(p - d),
         hi = S.upper_bound(p + d);
    for (; lo != hi; ++lo)
      ret = min(ret, {(*lo - p).sq(), {*lo, p}});
    S.insert(p);
  }
  return ret.second;
}
```

8.10 Convex Hull

```
/**
 * Descripcion: encuentra la envolvente convexa de un
 * conjunto de puntos dados. Una envolvente convexa es la
 * minima region convexa que contiene a todos los puntos
 * del conjunto.
 * Tiempo: O(n log n)
 */

vector<Point> convexHull(vector<Point> pts) {
  if (SZ(pts) <= 1) return pts;
  sort(ALL(pts));
  vector<Point> h(SZ(pts) + 1);
  int s = 0, t = 0;
  for (int it = 2; it--; s = --t, reverse(ALL(pts)))
    for (Point p : pts) {
      while (t >= s + 2 &&
             h[t - 2].cross(h[t - 1], p) <= 0)
        t--;  // quitar = si se incluye colineares
      h[t++] = p;
    }
  return {h.begin(),
          h.begin() + t - (t == 2 && h[0] == h[1])};
}
```

8.11 Half Plane

```
/*
 * Descripcion: Dado un conjunto de semiplanos calcula la
 * interseccion de estos representandolos en un poligono
 * convexo. Donde cada punto dentro del poligono esta
 * dentro de todos los semiplanos
 * - Cada semiplano apunta en su region izquierda
 * - Se asume que no hay semiplanos paralelos
 *
 * Tiempo: O(N Log N)
 */

typedef double T;
const double INF = 1e9;

struct HalfPlane {
  Point p, pq;
  T angle;
  HalfPlane() {}
  HalfPlane(Point _p, Point _q)
      : p(_p), pq(_q - _p), angle(atan2(pq.y, pq.x)) {}
  bool operator<(HalfPlane b) const {
    return less(angle, b.angle);
  }
  bool out(Point q) {
    return lessEqual(pq.cross(q - p), 0);
  }
  Point intersect(HalfPlane l) {
    if (equal(pq.cross(l.pq), 0)) return Point{INF, INF};
    return l.p +
           l.pq * ((p - l.p).cross(pq) / l.pq.cross(pq));
  }
};

vector<Point> intersect(vector<HalfPlane> b) {
  vector<Point> bx = {Point{INF, INF}, Point{-INF, INF},
                      Point{-INF, -INF},
                      Point{INF, -INF}};
  for (int i = 0; i < 4; ++i)
    b.emplace_back(bx[i], bx[(i + 1) % 4]);
  sort(b.begin(), b.end());
  int n = b.size(), q = 1, h = 0;
  vector<HalfPlane> c(b.size() + 10);
  for (int i = 0; i < n; ++i) {
    while (q < h && b[i].out(c[h].intersect(c[h - 1])))
      h--;
    while (q < h && b[i].out(c[q].intersect(c[q + 1])))
      q++;
    c[++h] = b[i];
    if (q < h && equal(c[h].pq.cross(c[h - 1].pq), 0)) {
      if (lessEqual(c[h].pq.dot(c[h - 1].pq), 0))
        return {};
      h--;
      if (b[i].out(c[h].p)) c[h] = b[i];
    }
  }
  while (q < h - 1 && c[q].out(c[h].intersect(c[h - 1])))
    h--;
  while (q < h - 1 && c[h].out(c[q].intersect(c[q + 1])))
    q++;
  if (h - q <= 1) return {};
  c[h + 1] = c[q];
  vector<Point> s;
  for (int i = q; i < h + 1; ++i)
```

```
    s.push_back(c[i].intersect(c[i + 1]));
  return s;
}
```

8.12 Line

```
typedef ll T;
struct Line {
  Point v;
  T c;

  // De vector direccional v y offset c
  Line(Point v, T c) : v(v), c(c) {}

  // De la ecuacion ax+by=c
  Line(T a, T b, T c) : v({b, -a}), c(c) {}

  // De punto P a punto Q
  Line(Point p, Point q) : v(q - p), c(v.cross(p)) {}

  // 0 si se encuentra en la linea, > 0 arriba, < 0 abajo
  T side(Point p) { return v.cross(p) - c; }

  double dist(Point p) { return abs(side(p)) / v.norm(); }
  double sqDist(Point p) {
    return side(p) * side(p) / (double)v.sq();
  }  // si se trabaja con enteros
  Line perp(Point p) { return {p, p + v.perp()}; }
  Line translate(Point t) { return {v, c + v.cross(t)}; }
  Line shiftLeft(double dist) {
    return {v, c + dist * v.norm()};
  }

  Point proj(Point p) {
    return p - v.perp() * side(p) / v.sq();
  }  // Punto en linea mas cercano a P
  Point refl(Point p) {
    return p - v.perp() * 2 * side(p) / v.sq();
  }

  // Sirve para comparar si un punto A esta antes de B en
  // una linea
  bool cmpProj(Point p, Point q) {
    return v.dot(p) < v.dot(q);
  }
};

bool areParallel(Line l1, Line l2) {
  return (l1.v.cross(l2.v) == 0);
}
bool areIntersect(Line l1, Line l2, Point& p) {
  T d = l1.v.cross(l2.v);
  if (d == 0)
    return false;  // cambiar a epsilon si es double
  p = (l2.v * l1.c - l1.v * l2.c) / d;  // requiere double
  return true;
}

// Un angulo bisector de dos lineas es una linea que forma
// angulos iguales con l1 y l2
Line bisector(Line l1, Line l2, bool interior) {
  assert(l1.v.cross(l2.v) !=
         0);  // l1 y l2 no pueden ser paralelas
  double sign = interior ? 1 : -1;
  return {l2.v / l2.v.norm() + l1.v / l1.v.norm() * sign,
          l2.c / l2.v.norm() + l1.c / l1.v.norm() * sign};
}
```

8.13 Minimum Enclosing Circle

```
/*
 * Descripcion: calcula el circulo minimo que encierra
 * un conjunto de puntos con el algoritmo de Welzl
 *
 * Tiempo: esperado O(n)
 */

pair<Point, double> mec(vector<Point> pts) {
  shuffle(ALL(pts), mt19937(time(0)));
  Point o = pts[0];
  double r = 0, EPS = 1 + 1e-8;
  FOR(i, 0, SZ(pts)) if ((o - pts[i]).norm() > r * EPS) {
    o = pts[i], r = 0;
    FOR(j, 0, i) if ((o - pts[j]).norm() > r * EPS) {
      o = (pts[i] + pts[j]) / 2;
      r = (o - pts[i]).norm();
      FOR(k, 0, j) if ((o - pts[k]).norm() > r * EPS) {
        o = circumCenter(pts[i], pts[j], pts[k]);
        r = (o - pts[i]).norm();
      }
    }
  }
  return {o, r};
}
```

8.14 Minkowski Sum

```
/*
 * Descripcion: calcula la suma de Minkowski de dos
 * poligonos convexos (eliminar colineales primero). La
 * respuesta es retornada en orden CCW
 * Tiempo: esperado
 * O(|p| + |q|)
 */

void reorder(vector<Point>& p) {
  if (less(orient(p[0], p[1], p[2]), 0)) reverse(ALL(p));
  int pos = 0;
  FOR(i, 1, SZ(p))
  if (Point{p[i].y, p[i].x} < Point{p[pos].y, p[pos].x})
    pos = i;
  rotate(p.begin(), p.begin() + pos, p.end());
}
vector<Point> minkowski_sum(vector<Point> p,
                            vector<Point> q) {
  if (min(SZ(p), SZ(q)) < 3) {
    vector<Point> v;
    for (Point pp : p)
      for (Point qq : q) v.pb(pp + qq);
    return convexHull(v);
  }
  reorder(p);
  reorder(q);
  FOR(i, 0, 2) p.pb(p[i]), q.pb(q[i]);
  vector<Point> r;
  int i = 0, j = 0;
  while (i + 2 < SZ(p) || j + 2 < SZ(q)) {
    r.pb(p[i] + q[j]);
    auto cross = (p[i + 1] - p[i]).cross(q[j + 1] - q[j]);
    i += greaterEqual(cross, 0);
    j += lessEqual(cross, 0);
  }
  return r;
}
```

8.15 Point

```
constexpr double EPS =
    1e-9;  // 1e-9 es suficiente para problemas de
           // precision doble
constexpr double PI = acos(-1.0);

inline double DEG_to_RAD(double d) {
  return (d * PI / 180.0);
}
inline double RAD_to_DEG(double r) {
  return (r * 180.0 / PI);
}

using T = double;

int sgn(T x) {
  if (x < -EPS) return -1;
  if (x > EPS) return 1;
  return 0;
}
// Operaciones de comparacion para punto flotante
// Usar estas funciones en lugar de <, <=, >, >=, ==
// cuando trabajes con doubles y tal operacion te afecta
// el flujo del programa.
// Para respuestas max, min son suficientes
bool less(T a, T b) { return sgn(a - b) < 0; }
bool lessEqual(T a, T b) { return sgn(a - b) <= 0; }
bool greater(T a, T b) { return sgn(a - b) > 0; }
bool greaterEqual(T a, T b) { return sgn(a - b) >= 0; }
bool equal(T a, T b) { return sgn(a - b) == 0; }

struct Point {
  T x, y;

  // Operaciones Punto - Punto
  Point operator+(Point p) const {
    return {x + p.x, y + p.y};
  }
  Point operator-(Point p) const {
    return {x - p.x, y - p.y};
  }
  Point operator*(Point b) const {
    return {x * b.x - y * b.y, x * b.y + y * b.x};
  }

  // Operaciones Punto - Numero
  Point operator*(T d) const { return {x * d, y * d}; }
  Point operator/(T d) const {
    return {x / d, y / d};
  }  // Solo para punto flotante

  // Operaciones de comparacion para punto flotante
  bool operator<(Point p) const {
    return less(x, p.x) ||
           (equal(x, p.x) && less(y, p.y));
  }
  bool operator==(Point p) const {
    return equal(x, p.x) && equal(y, p.y);
  }
  bool operator!=(Point p) const { return !(*this == p); }

  // Operaciones de comparacion para enteros
  bool operator<(Point p) const {
    return tie(x, y) < tie(p.x, p.y);
  }
  bool operator==(Point p) const {
    return tie(x, y) == tie(p.x, p.y);
  }

  T sq() { return x * x + y * y; }
  double norm() { return sqrt(sq()); }
  Point unit() { return *this / norm(); }

  // Operaciones generales:
  Point translate(Point v) { return *this + v; }
  Point scale(Point c, double factor) {
    return c + (*this - c) * factor;
  }
  Point rotate(double ang) {
    return {x * cos(ang) - y * sin(ang),
            x * sin(ang) + y * cos(ang)};
  }
  Point rot_around(double ang, Point c) {
    return c + (*this - c).rotate(ang);
  }
  Point perp() { return {-y, x}; }
```

```
  T dot(Point p) { return x * p.x + y * p.y; }
  T cross(Point p) const { return x * p.y - y * p.x; }
  T cross(Point a, Point b) const {
    return (a - *this).cross(b - *this);
  }
  T angle() const {
    T ang = atan2(y, x);
    return ang < 0 ? ang + 2 * PI : ang;
  }

  friend ostream& operator<<(ostream& os, Point p) {
    return os << "(" << p.x << "," << p.y << ")";
  }
};

// Vector: p2-p1
double dist(Point p1, Point p2) {
  return hypot(p1.x - p2.x, p1.y - p2.y);
}
bool isPerp(Point v, Point w) {
  return equal(v.dot(w), 0);
}
//-1 -> left / 0 -> collinear / +1 -> right
T orient(Point a, Point b, Point c) {
  return a.cross(b, c);
}
bool cw(Point a, Point b, Point c) {
  return less(orient(a, b, c), 0);
}
bool ccw(Point a, Point b, Point c) {
  return greater(orient(a, b, c), 0);
}

// ANGULOS
// Para c++17
double angle(Point v, Point w) {
  return acos(
      clamp(v.dot(w) / v.norm() / w.norm(), -1.0, 1.0));
}
// C++14 o menor
double angle(Point v, Point w) {
  double cosTheta = v.dot(w) / v.norm() / w.norm();
  return acos(max(-1.0, min(1.0, cosTheta)));
}
// angulo aob
double angle(Point o, Point a, Point b) {
  return angle(a - o, b - o);
}
double orientedAngle(Point o, Point a, Point b) {
  if (ccw(o, a, b))
    return angle(a - o, b - o);
  else
    return 2 * PI - angle(a - o, b - o);
}
bool inAngle(Point o, Point a, Point b, Point p) {
  assert(orient(o, a, b) != 0);
  if (cw(o, a, b)) swap(b, c);
  return ccw(o, a, p) && cw(o, c, p);
}
```

8.16 Polar Sort

```
/*
 * Descripcion: ordena los puntos segun el angulo.
 * Comienza a partir de la izquierda en contra de las
 * manecillas
 */

int half(Point p) {
  return p.y > 0 || (p.y == 0 && p.x < 0);
}

// Pro-tip: si los puntos se encuentran en la misma
// direccion son considerados iguales, entonces se
// ordenaran arbitrariamente. Si se busca un desempate, se
// puede usar la magnitud sq(v)
void polarSort(vector<Point> &v) {
  sort(ALL(v), [](Point v, Point w) {
    return make_tuple(half(v), 0) <
           make_tuple(half(w), v.cross(w));
  });
}

void polarSortAround(Point o, vector<Point> &v) {
  sort(ALL(v), [](Point v, Point w) {
    return make_tuple(half(v - o), 0) <
           make_tuple(half(w - o), (v - o).cross(w - o));
  });
}

// Si se quiere modificar que el primer angulo del polar
// sort sea el vector v utilizar esta implementacion
Point v = {/* el que sea menos {0,0} */};
bool half(Point p) {
  return v.cross(p) < 0 ||
         (v.cross(p) == 0 && v.dot(p) < 0);
}
```

8.17 Polygon

```
// Retorna el area de un triangulo
double areaTriangle(Point a, Point b, Point c) {
  return abs((b - a).cross(c - a)) / 2.0;
}

// Retorna si el punto esta dentro del triangulo
bool pointInTriangle(Point a, Point b, Point c, Point p) {
  T s1 = abs(a.cross(b, c));
  T s2 = abs(p.cross(a, b)) + abs(p.cross(b, c)) +
         abs(p.cross(c, a));
  return s1 == s2;
}

// Retorna el area del poligono
double areaPolygon(vector<Point> p) {
  double area = 0.0;
  int n = SZ(p);
  FOR(i, 0, n) { area += p[i].cross(p[(i + 1) % n]); }
  return abs(area) / 2.0;
}

// Retorna si el poligono es convexo
bool isConvex(vector<Point> p) {
  bool hasPos = false, hasNeg = false;
  for (int i = 0, n = SZ(p); i < n; i++) {
    int o = orient(p[i], p[(i + 1) % n], p[(i + 2) % n]);
    if (o > 0) hasPos = true;
    if (o < 0) hasNeg = true;
  }
  return !(hasPos && hasNeg);
}

// Retorna 1/0/-1 si el punto p esta dentro/sobre/fuera de
// cualquier poligono P concavo/convexo
//
 * Tiempo: O(n)
int inPolygon(vector<Point> poly, Point p) {
  int n = SZ(poly), ans = 0;
  FOR(i, 0, n) {
    Point p1 = poly[i], p2 = poly[(i + 1) % n];
    if (p1.y > p2.y) swap(p1, p2);
    if (onSegment(p1, p2, p)) return 0;
    ans ^= (p1.y <= p.y && p.y < p2.y &&
            p.cross(p1, p2) > 0);
  }
  return ans ? -1 : 1;
}

// Retorna el centroide del poligono
Point polygonCenter(vector<Point>& v) {
  Point res{0, 0};
  double A = 0;
  for (int i = 0, j = SZ(v) - 1; i < SZ(v); j = i++) {
    res = res + (v[i] + v[j]) * v[j].cross(v[i]);
    A += v[j].cross(v[i]);
  }
  return res / A / 3;
}

// Determina si un punto P se encuentra dentro de un
// poligono convexo ordenado en ccw y sin puntos
// colineares (Convex hull) Tiempo O(log n)
bool inPolygonCH(vector<Point>& l, Point p,
                 bool strict = true) {
  int a = 1, b = SZ(l) - 1, r = !strict;
  if (SZ(l) < 3) return r && onSegment(l[0], l.back(), p);
  if (orient(l[0], l[a], l[b]) > 0) swap(a, b);
  if (orient(l[0], l[a], p) >= r ||
      orient(l[0], l[b], p) <= -r)
    return false;
  while (abs(a - b) > 1) {
    int c = (a + b) / 2;
    (orient(l[0], l[c], p) > 0 ? b : a) = c;
  }
  return sgn(l[a].cross(l[b], p)) < r;
}

// Retorna los dos puntos con mayor distancia en un
// poligono convexo ordenado en ccw y sin puntos
// colineares (Convex hull) Tiempo O(n)
array<Point, 2> hullDiameter(vector<Point> S) {
  int n = SZ(S), j = n < 2 ? 0 : 1;
  pair<ll, array<Point, 2>> res({0, {S[0], S[0]}});
  FOR(i, 0, j) {
    for (;; j = (j + 1) % n) {
      res = max(res, {(S[i] - S[j]).sq(), {S[i], S[j]}});
      if ((S[(j + 1) % n] - S[j])
            .cross(S[i + 1] - S[i]) >= 0)
        break;
    }
  }
  return res.second;
}

// Retorna el poligono que se encuentra a la izquierda de
// la linea que va de s a e despues del corte
vector<Point> polygonCut(vector<Point>& poly, Point s,
                         Point e) {
  vector<Point> res;
  FOR(i, 0, SZ(poly)) {
    Point cur = poly[i],
          prev = i ? poly[i - 1] : poly.back();
    bool side = s.cross(e, cur) < 0;
    if (side != (s.cross(e, prev) < 0)) {
      Point p;
      areIntersect(Line(s, e), Line(cur, prev), p);
      res.push_back(p);
    }
    if (side) res.push_back(cur);
  }
  return res;
}
```

8.18 Segment

```
// Retorna si el Punto P se encuentra dentro del circulo
// entre A y B
bool inDisk(Point a, Point b, Point p) {
  return (a - p).dot(b - p) <= 0;
}

// Retorna si el punto P se encuentra en el segmento de
// puntos S a E
bool onSegment(Point a, Point b, Point p) {
```

```
  return a.cross(b, p) == 0 && inDisk(a, b, p);
}

// SEGMENTO - SEGMENTO INTERSECCION
bool properInter(Point a, Point b, Point c, Point d,
                 Point& p) {
  double oa = orient(c, d, a), ob = orient(c, d, b),
         oc = orient(a, b, c), od = orient(a, b, d);
  if (oa * ob < 0 && oc * od < 0) {
    p = (a * ob - b * oa) / (ob - oa);
    return true;
  }
  return false;
}
// Si existe un punto de interseccion unico entre los
// segmentos de linea que van de A a B y de C a D, se
// devuelve. Si no existe ningun punto de interseccion, se
// devuelve un vector vacio. Si existen infinitos, se
// devuelve un vector con 2 elementos, que contiene los
// puntos finales del segmento de linea comun.
vector<Point> segInter(Point a, Point b, Point c,
                       Point d) {
  Point p;
  if (properInter(a, b, c, d, p)) return {p};
  set<Point> s;
  if (onSegment(c, d, a)) s.insert(a);
  if (onSegment(c, d, b)) s.insert(b);
  if (onSegment(a, b, c)) s.insert(c);
  if (onSegment(a, b, d)) s.insert(d);
  return {ALL(s)};
}

// SEGMENTO - PUNTO DISTANCIA
double segPoint(Point a, Point b, Point p) {
  if (a != b) {
    Line l(a, b);
    if (l.cmpProj(a, p) && l.cmpProj(p, b))
      return l.dist(p);
  }
  return min((p - a).norm(), (p - b).norm());
}

// SEGMENTO - SEGMENTO DISTANCIA
double segSeg(Point a, Point b, Point c, Point d) {
  Point dummy;
  if (properInter(a, b, c, d, dummy)) return 0;
  return min({segPoint(a, b, c), segPoint(a, b, d),
              segPoint(c, d, a), segPoint(c, d, b)});
}
```

9 Extras

9.1 Custom Hash

```cpp
/**
 * Descripcion: Hash Maps/Sets ultra-rapidos (PBDS)
 * resistentes a hacks. Filosofia: Key Compression. Nunca
 * usar structs (pair, vector) como llaves. Codificarlas a
 * uint64_t o u128 usando Encoder antes de insertar.
 * Uso:
 * FastMap<int> mp; mp.resize(N);        // Mapa de
 * uint64_t -> int uint64_t k = Encoder::p64(x, y); //
 * Comprimir par (x, y) o punto 2D mp[k]++; // O(1)
 * amortizado FastSet128 st;                    // Set
 * de 128 bits (Doble Hash) u128 k2 = Encoder::s128(vec);
 * // Comprimir vector/string st.insert(k2); Notas:
 * - FastMap (gp): Usa Probing. Muy rapido, mas memoria.
 * - MemMap (cc) : Usa Chaining. Mas lento, menos memoria
 * (usar en limites < 128MB).
 * - resize(N)   : Evita rehash costosos. Usar siempre si
 * N > 10^5.
 * - clear()     : O(capacidad). En multi-casos, declarar
 * mapa adentro del while(t--).
 * Tiempo: O(1) amortizado
 * por operacion.
 */

#include <bits/stdc++.h>

#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>

using namespace std;
using namespace __gnu_pbds;

using u128 = __uint128_t;
static const uint64_t SEED = chrono::steady_clock::now()
                                .time_since_epoch()
                                .count();

struct SafeHash {
  static uint64_t sm64(uint64_t x) {
    x += 0x9e3779b97f4a7c15;
    x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
    x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
    return x ^ (x >> 31);
  }
  size_t operator()(uint64_t x) const {
    return sm64(x + SEED);
  }
  size_t operator()(u128 x) const {
    uint64_t t = (uint64_t)(x >> 64), b = (uint64_t)x;
    return sm64(t ^ b ^ SEED);
  }
};

template <class K, class V, class H>
using gp_base = gp_hash_table<
    K, V, H, equal_to<K>, direct_mask_range_hashing<>,
    linear_probe_fn<>,
    hash_standard_resize_policy<
        hash_exponential_size_policy<>,
        hash_load_check_resize_trigger<>, true>>;

template <class K, class V, class H>
using cc_base = cc_hash_table<
    K, V, H, equal_to<K>, direct_mask_range_hashing<>,
    hash_standard_resize_policy<
        hash_exponential_size_policy<>,
        hash_load_check_resize_trigger<>, true>>;

template <class V>
using FastMap = gp_base<uint64_t, V, SafeHash>;
template <class V>
using MemMap = cc_base<uint64_t, V, SafeHash>;
using FastSet = FastMap<null_type>;

template <class V>
using FastMap128 = gp_base<u128, V, SafeHash>;
using FastSet128 = FastMap128<null_type>;

namespace Encoder {
inline uint64_t c64(uint64_t h, uint64_t v) {
  return h ^ (v + 0x9e3779b97f4a7c15ULL + (h << 6) +
              (h >> 2));
}
template <class T1, class T2>
uint64_t p64(T1 a, T2 b) {
  return c64(hash<T1>{}(a), hash<T2>{}(b));
}
template <class I>
uint64_t s64(const I& s) {
  uint64_t h = 0;
  for (const auto& x : s)
    h = c64(h, hash<typename I::value_type>{}(x));
  return h;
}
template <class T1, class T2>
u128 p128(T1 a, T2 b) {
  uint64_t h1 = p64(a, b),
           h2 = c64(hash<T2>{}(b) + 0xABCDE,
                    hash<T1>{}(a));
  return ((u128)h1 << 64) | h2;
}
inline u128 pack(uint64_t h1, uint64_t h2) {
  return ((u128)h1 << 64) | h2;
}
}  // namespace Encoder
```

9.2 Dates

```cpp
/**
 * Descripcion: rutinas para realizar calculos sobre
 * fechas, en estas rutinas, los meses son expresados como
 * enteros desde el 1 al 12, los dias como enteros desde
 * el 1 al 31, y los anios como enteros de 4 digitos.
 */
string dayOfWeek[] = {"Mon", "Tue", "Wed", "Thu",
                      "Fri", "Sat", "Sun"};

// Convierte fecha Gregoriana a entero (fecha Juliana)
int dateToInt(int m, int d, int y) {
  return 1461 * (y + 4800 + (m - 14) / 12) / 4 +
         367 * (m - 2 - (m - 14) / 12 * 12) / 12 -
         3 * ((y + 4900 + (m - 14) / 12) / 100) / 4 + d -
         32075;
}

// Convierte entero (fecha Juliana) a Gregoriana: M/D/Y
void intToDate(int jd, int &m, int &d, int &y) {
  int x, n, i, j;

  x = jd + 68569;
  n = 4 * x / 146097;
  x -= (146097 * n + 3) / 4;
  i = (4000 * (x + 1)) / 1461001;
  x -= 1461 * i / 4 - 31;
  j = 80 * x / 2447;
  d = x - 2447 * j / 80;
  x = j / 11;
  m = j + 2 - 12 * x;
  y = 100 * (n - 49) + i + x;
}

// Convierte entero (fecha Juliana) a dia de la semana
string intToDay(int jd) { return dayOfWeek[jd % 7]; }

int main() {
  int jd = dateToInt(3, 24, 2004);
  int m, d, y;
  intToDate(jd, m, d, y);
  string day = intToDay(jd);
  // Salida esperada:
  // 2453089
  // 3/24/2004
  // Wed
  cout << jd << endl
       << m << "/" << d << "/" << y << endl
       << day << endl;
}
```

9.3 Iterate Submasks

```cpp
for (int msk = 0; msk < MX_MSK; msk++) {
  for (int submsk = msk; submsk;
       submsk = (submsk - 1) & msk) {
  }
}
```

9.4 LIS

```cpp
/**
 * Descripcion: longitud de la subsecuencia estrictamente
 * creciente mas larga (LIS). d[k] es el menor valor en el
 * que puede terminar una subsecuencia creciente de
 * longitud k + 1. Para no decreciente usar upper_bound en
 * vez de lower_bound. Reconstruccion: guardar p[i] =
 * posicion donde se coloco a[i] en d. Recorrer de derecha
 * a izquierda tomando el primer i con p[i] = k, luego el
 * primero con p[i] = k - 1 y a[i] < ultimo tomado, etc.
 *
 * Tiempo: O(n log n)
 */

int lis(const vi& a) {
  vi d;
  for (int x : a) {
    auto it = lower_bound(ALL(d), x);
    if (it == end(d))
      d.pb(x);
    else
      *it = x;
  }
  return SZ(d);
}
```

9.5 Random

```cpp
/*
 * Descripcion: Basado en Mersenne Twister genera un
 * numero pseudoaleatorio de 32 bits mucho mas rapido que
 * la funcion rand() de c++.
 * Si necesita hasta 64 bits usar mt19937_64 en su lugar
 */

mt19937 rng(chrono::steady_clock::now()
                .time_since_epoch()
                .count());

// Aplicaciones

// Mezclar un arreglo
shuffle(ALL(a), rng);

// Generar un numero entero en el intervalo [l,r]
uniform_int_distribution<int>(l, r)(rng)
```